



Synopsys IP

Adaptive Voltage Scaling Bus (AVSBus) Controller

Databook

Product Code: I864-0

Copyright Notice and Proprietary Information

© 2025 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History	7
Preface	9
Related Product Documentation	9
Web Resources	9
Reference Documentation	10
Synopsys Statement on Inclusivity and Diversity	10
Customer Support	10
Chapter 1 Product Overview	13
1.1. General Product Description	14
1.2. Standards Compliance	15
1.3. Features	16
1.4. Area and Power Numbers	17
1.5. Controller Requirements	18
1.6. Deliverables	19
Chapter 2 Architecture	21
2.1. Transport Topology	23
2.2. Overview	24
2.3. Frame Alignment	27
2.4. Supported Command Data Types	28
2.5. AVSBus Clock Generation	29
2.5.1. Registers Related to AVSBus Clock Generation Feature	29
2.6. Idle Bus Condition	30
2.7. Target Acknowledgement	31
2.8. Status Response Frame	32
2.8.1. Interrupt Related to Status Response Frame	33
2.9. Target Resynchronization	34
2.9.1. Overview of Target Resynchronization Feature	34
2.9.2. Description of Target Resynchronization Feature	34
2.9.3. Signals Related to Target Resynchronization Feature	34

2.10. Cyclic Redundancy Check (CRC) Verification Feature.....	36
2.10.1. Overview of CRC Verification Feature.....	36
2.10.2. Description of CRC Verification Feature.....	36
2.10.3. Pause on CRC Error.....	37
2.10.4. Signals Related to CRC Verification Feature.....	39
2.11. Communication from the AVSBus Target to the AVSBus Controller.....	40
2.11.1. Interrupts Related to Communication from the AVSBus Target to the AVSBus Controller.....	40
2.12. Propagation Delay Management.....	41
2.12.1. Handling the Propagation Delay.....	43
2.12.2. Registers for Handling Propagation Delay.....	43
2.13. Transmit and Receive FIFO Buffers.....	44
2.14. Soft Reset Operation.....	46
2.14.1. Overview of Soft Reset Operation.....	46
2.14.2. Description of Soft Reset Operation.....	46
2.14.3. Registers Related to Soft Reset Operation.....	46
2.15. Operation of Interrupt Registers.....	47
2.15.1. Selecting Type and Polarity of Interrupts.....	47
2.15.2. Interrupt Signals.....	48
2.15.3. Interrupt Related Registers.....	48
Chapter 3 Interfaces.....	49
3.1. APB Completer Interface.....	50
3.1.1. APB Wait Cycle Insertion.....	50
3.1.2. APB Write Strobe.....	51
3.1.3. APB Protection.....	51
3.1.4. APB Error Response.....	51
3.1.5. Signals Related to APB Completer Interface.....	53
3.2. DMA Handshaking Interface.....	54
3.2.1. Overview of Operation.....	54
3.2.2. Transmit Watermark Level and Transmit FIFO Underflow.....	56
3.2.3. Choosing the Transmit Watermark Level.....	56
3.2.4. Selecting DST_MSIZ and Transmit FIFO Overflow.....	58
3.2.5. Receive Watermark Level and Receive FIFO Overflow.....	58
3.2.6. Choosing the Receive Watermark level.....	58
3.2.7. Selecting SRC_MSIZ and Receive FIFO Underflow.....	58

3.2.8. Handshaking Interface Operation.....59

3.2.9. Enabling the DMA Handshaking Interface..... 62

3.2.10. Signals Related to DMA Handshaking Interface.....62

3.2.11. Registers Related to DMA Handshaking Interface..... 62

3.3. External Memory Interface.....63

3.3.1. Enabling External Memory..... 63

3.3.2. Signals Related to External Memory Interface.....63

3.4. Debug Interface..... 65

3.4.1. Signals Related to Debug Interface Feature.....65

Appendix A Area and Power..... 67

Revision History

This table shows the revision history for the databook from release to release.

Version	Date	Description
1.00a-lca00	May 2025	First version

Preface

This chapter contains the following sections:

- “[Related Product Documentation](#)” on page 9
- “[Web Resources](#)” on page 9
- “[Reference Documentation](#)” on page 10
- “[Synopsys Statement on Inclusivity and Diversity](#)” on page 10
- “[Customer Support](#)” on page 10

Related Product Documentation

- [Synopsys Controller IP Adaptive Voltage Scaling Bus \(AVSBus\) User Guide](#)
- [Synopsys Controller IP Adaptive Voltage Scaling Bus \(AVSBus\) Installation Guide](#)
- [Synopsys Controller IP Adaptive Voltage Scaling Bus \(AVSBus\) Reference Manual](#)
- [Synopsys Controller IP Adaptive Voltage Scaling Bus \(AVSBus\) Release Notes](#)

Web Resources

For more information, check the following Synopsys online resources:

- Synopsys IP product information: <https://www.synopsys.com/designware-ip.html>
- Your custom Synopsys IP page: <https://www.synopsys.com/dw/mydesignware.php>

- Documentation through SolvNetPlus: <https://solvnetplus.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL):
<https://www.synopsys.com/support/licensing-installation-computeplatforms/licensing.html>

Reference Documentation

The following references contain useful information concerning the protocols addressed by DWC_avsbus:

- AMBA 4 APB Protocol Specification, Revision 2.0, ARM for APB4 interface
- PMBUS Power System Management Protocol Specification Part III - AVSBus, Revision 1.4.1

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Customer Support

To obtain support for your product, prepare the required files and contact the support center using one of the methods described:

- Prepare the following debug information, if applicable:
 - For environment set-up problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, select the following menu:

- **File > Build Debug Tar-file**

Check all the boxes in the dialog box that apply to your issue. This option gathers all the Synopsys product data needed to begin debugging an issue and writes it to the *<core tool startup directory>/debug.tar.gz* file.

- For simulation issues outside of coreConsultant or coreAssembler:
 - Create a waveform file (such as VPD, VCD, or FSDB).
 - Identify the hierarchy path to the Design Under Test (DUT).
 - Identify the timestamp of any signals or locations in the waveforms that are not understood.
- For the fastest response, enter a case through SolvNetPlus:
 - a. <https://solvnetplus.synopsys.com>



Note SolvNetPlus does not support Internet Explorer.

- b. Click the **Cases** menu and then click **Create a New Case** (below the list of cases).

c. Complete the mandatory fields that are marked with an asterisk and click **Save**.

Make sure to include the following:

- **Product L1:** DesignWare Library IP
- **Product L2:** AMBA

d. After creating the case, attach any debug files you created.

For more information about general usage information, refer to the following article in SolvNetPlus:

<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>

■ Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):

- ☐ Include the Product L1 and Product L2 names, and Version number in your e-mail so it can be routed correctly.
- ☐ For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
- ☐ Attach any debug files you created.

■ Or, telephone your local support center:

- ☐ North America:

Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.

- ☐ All other countries:

<https://www.synopsys.com/support/global-support-centers.html>

1

Product Overview

This chapter contains the following sections:

- [“General Product Description”](#) on page 14
- [“Standards Compliance”](#) on page 15
- [“Features”](#) on page 16
- [“Area and Power Numbers”](#) on page 17
- [“Controller Requirements”](#) on page 18
- [“Deliverables”](#) on page 19

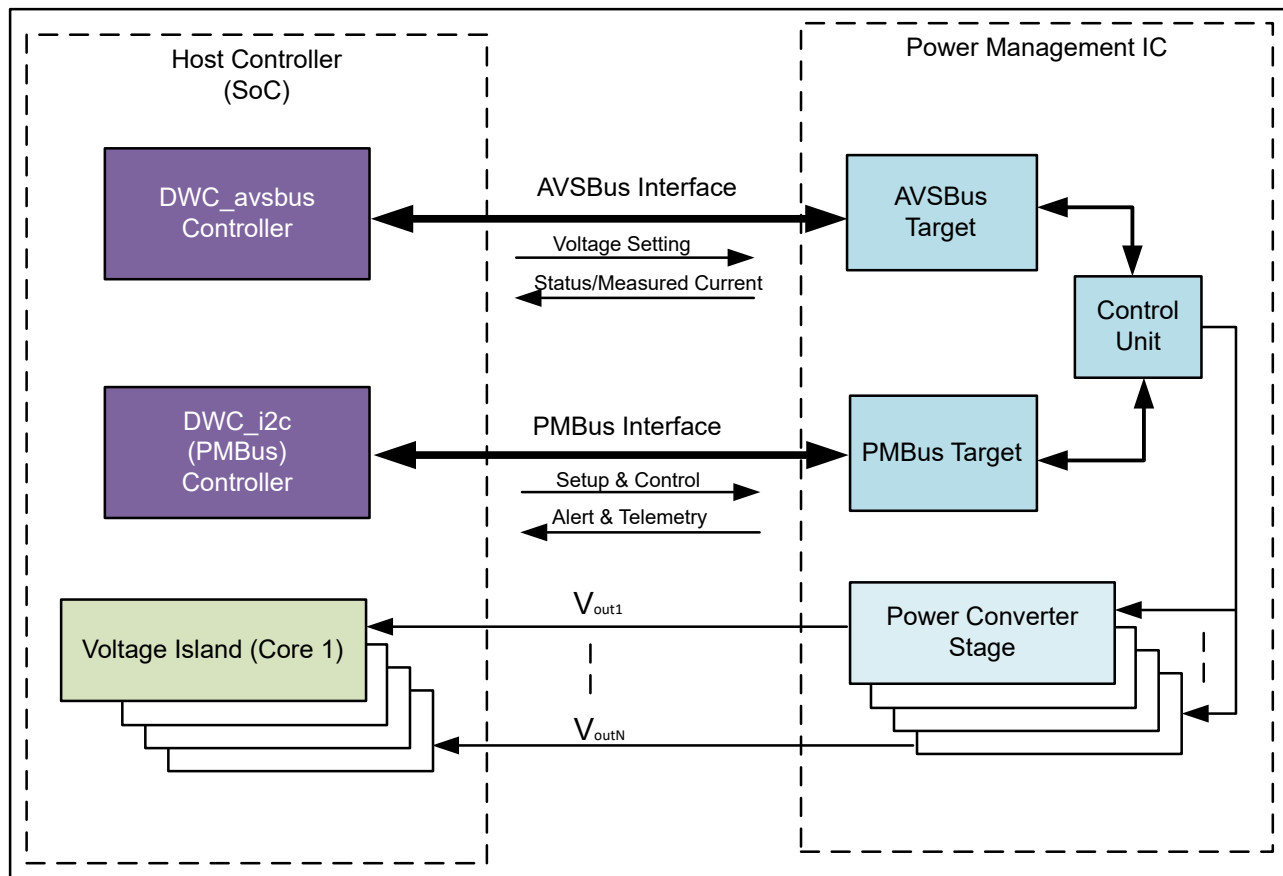
1.1 General Product Description

Adaptive Voltage Scaling is designed to enable a device to achieve faster and direct control of power supply voltages to minimize the power consumption for a given operating condition. The AVSBus protocol covers a wide range of power system architectures and converters. The AVSBus is a 3-wire interface protocol to provide bidirectional communication between one controller and one target for controlling voltage scaling. AVSBus controller can be used stand-alone or as a complement to the PMBus. AVSBus is an extension of PMBus. When PMBus is integrated with AVSBus, PMBus can be used to initialize setup and handover the control to AVSBus for setting rail voltages and monitoring rail status.

Figure 1-1 illustrates the integration of PMBus and AVSBus in a system.

Synopsys AVSBus Controller is a configurable, synthesizable, and programmable IP that enables point-to-point communication interface between a Point of Load (POL) voltage controller and its load for the purpose of Adaptive Voltage Scaling.

Figure 1-1 DWC_avsbus Block Diagram



1.2 Standards Compliance

DWC_avsbus compliants with the following specifications:

- PMBus Power System Management Protocol Specification Part III – AVSBus, Revision 1.4.1
- AMBA APB Protocol Specification, Revision 2.0, APB4 interface

DWC_avsbus implements AMBA APB completer interface as the host controller interface to CPU and perform software programming.

1.3 Features

AVSBus controller supports the following features:

General Features

- Soft reset
- Synchronous and asynchronous clock modes
- Configurable synchronizer depth
- DMA handshaking interface
- External memory interface
- Configurable transmit and receive buffer depth
- Debug interface
- Interrupt interface and interrupt status for better management of firmware or software
- Error management capabilities and error status reporting

Completer Interface Features

- Simple software interface consistent with Synopsys IP APB peripherals
- APB4 completer interface
- Access protection

AVSBus Features

- 3-Wire mode
- Target resynchronization
- Target alert detection
- CRC verification for AVSBus frames
- Clock suspension
- Programmable receive data sampling delay

1.4 Area and Power Numbers

For information on power and area requirements in the DWC_avsbus, see [Appendix A, “Area and Power”](#)

1.5 Controller Requirements

Refer to the *Synopsys Controller IP Adaptive Voltage Scaling Bus (AVSBus) User Guide* for information on clocks, and reset requirements.

1.6 Deliverables

The DWC_avsbus controller is packaged as a .run file. The license file, required to use the Synopsys DesignWare IP, is delivered separately. Use the Synopsys coreConsultant tool to configure, synthesize, and simulate the DWC_avsbus controller.

The DWC_avsbus controller image includes the following:

- Verilog RTL source code.
- A Packaged Verification Environment (PVE) in UVM
 - Allows you to re-execute the example tests on your controller configuration for the purpose of sanity checking your generated RTL in coreConsultant environment before moving the RTL to your SoC level verification.
 - Demonstrates connectivity and many types of transfers.
 - Is not intended for extensive verification.



The VC VIP library license shipped with the DWC_avsbus controller is intended only to verify the controller interfaces in a specific controller configuration out-of-the-box in the coreConsultant, to confirm that the controller meets the requirements. Do not use this VC VIP library license as a full-featured verification model at the subsystem or SoC level.

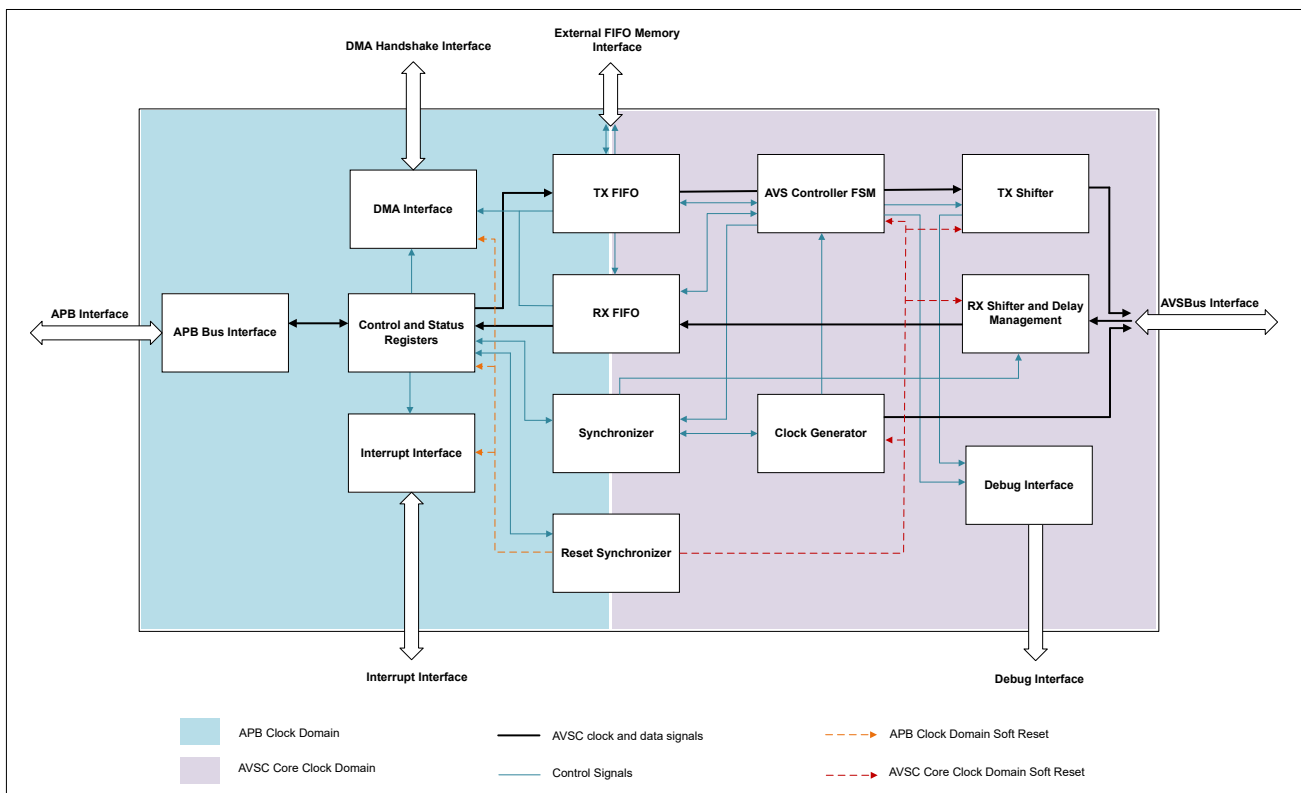
- Synthesis scripts for Synopsys Design Compiler, Fusion Compiler, and Synplicity Synplify tools.
- Regression scripts for running simulation with Synopsys VCS simulator.
- Gate-level validation scripts for:
 - Formal verification (using Formality)
 - Static timing analysis (using PrimeTime)
 - ATPG patterns verification (using TetraMax)
- SpyGlass and VC Spyglass checker rules for linting, CDC, and RDC.
- DWC_avsbus controller databook, user guide, installation guide, and release notes.
- Application Programming Interface (API) in C source code is available as a separate deliverable. (If needed, contact Synopsys for more information.)

2

Architecture

Figure 2-1 shows the top level architecture of the DWC_avsbus controller

Figure 2-1 DWC_avsbus System Level Block Diagram



This chapter contains the following sections:

- “Transport Topology” on page 23
- “Overview” on page 24
- “Frame Alignment” on page 27
- “Supported Command Data Types” on page 28
- “AVSBus Clock Generation” on page 29

- [“Idle Bus Condition” on page 30](#)
- [“Target Acknowledgement” on page 31](#)
- [“Status Response Frame” on page 32](#)
- [“Target Resynchronization” on page 34](#)
- [“Cyclic Redundancy Check \(CRC\) Verification Feature” on page 36](#)
- [“Communication from the AVSBus Target to the AVSBus Controller” on page 40](#)
- [“Propagation Delay Management” on page 41](#)
- [“Transmit and Receive FIFO Buffers” on page 44](#)
- [“Soft Reset Operation” on page 46](#)
- [“Operation of Interrupt Registers” on page 47](#)

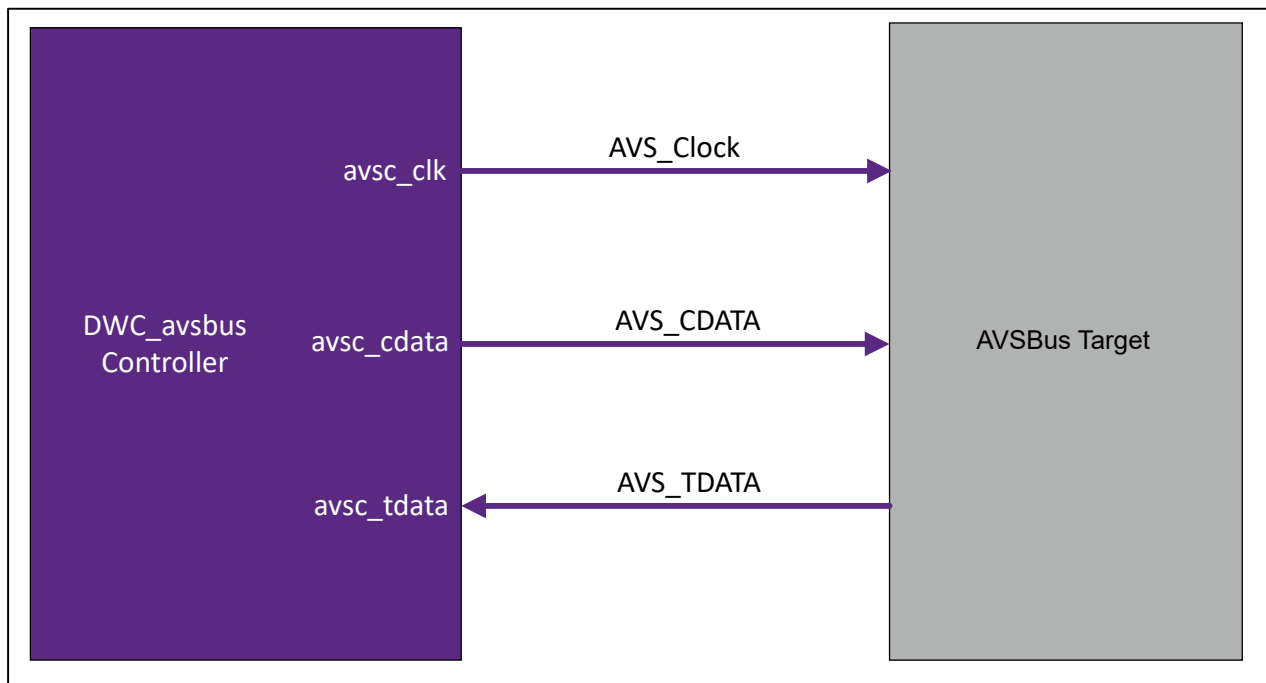
2.1 Transport Topology

DWC_avsbus supports 3-wire mode of operation to provide bidirectional communication between one powered device and one target for controlling voltage scaling. The 3-wire mode is the complete implementation of the AVSBus that allows a controller to:

- Receive acknowledgements from targets in response to write operations
- Read back data and configuration from targets
- Receive status response in every frame

Figure 2-2 shows the connection diagram of a typical 3-wire mode AVSBus implementation.

Figure 2-2 DWC_avsbus 3-Wire Mode Interfacing with AVSBus Target



- The **avsc_cdata** signal is driven by the AVSBus controller and carries data to the target.
- The **avsc_tdata** signal is driven by an AVSBus target and carries data to the controller.
- The **avsc_clk** signal is driven by the controller and clocks data for both the **avsc_cdata** and **avsc_tdata** lines.

2.2 Overview

The AVSBus protocol defines 64-bit long frames in the 3-wire mode of the complete implementation of AVSBus protocol. This communication protocol consists of two types of frame layouts:

- Write frame: The controller initiates the transfer with a 32-bit command, and the target responds with a 32-bit status frame.
- Read frame: Similar to the write frame, the controller sends a 32-bit command requesting data, and the target responds with a 32-bit data frame.

Each frame is composed of two sub-frames:

- The controller sub-frame, which always begins with a start code condition, and carries the data from the controller to the target using the `avsc_cdata` signal.
- The target sub-frame carries the data from the target to controller using the `avsc_tdata` signal.

The two sub-frames of a frame are related to each other as defined:

- A controller write sub-frame is immediately followed by a target write sub-frame.
- A controller read sub-frame is immediately followed by a target read sub-frame.

Therefore, the two sub-frames of an AVSBus transmit sequentially one after the other in 32-bit frames. This is the default behavior of the AVSBus frames. The frame format of these AVSBus frames is shown in [Figure 2-3](#) and [Figure 2-4](#) for write transfer and [Figure 2-5](#) and [Figure 2-6](#) for read transfer.

Figure 2-3 AVSBus Write Frame Structure

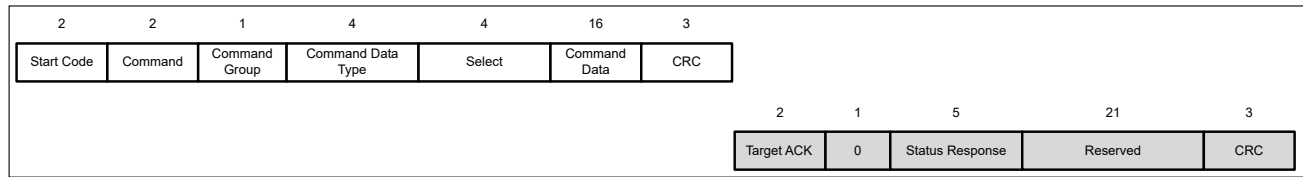


Figure 2-4 AVSBus Write Frame Over DWC_avsbus Interface

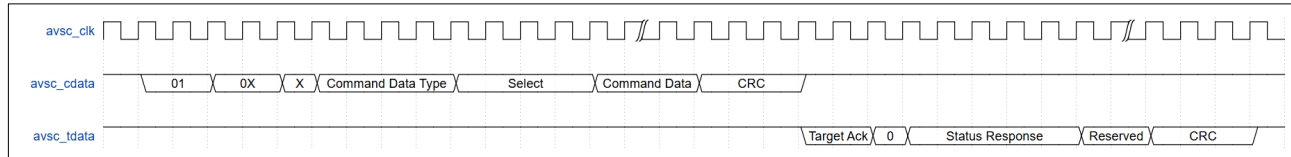


Figure 2-5 AVSBus Read Frame Structure

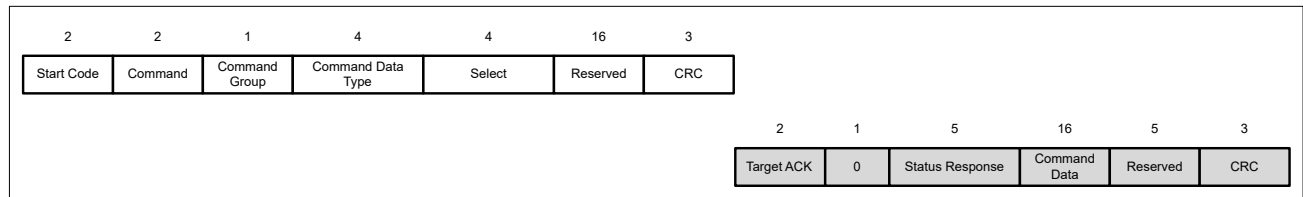
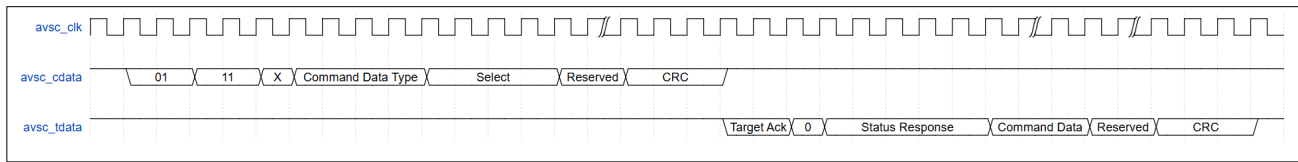


Figure 2-6 AVSBus Read Frame Over DWC_avsbus Interface

A Controller write/read sub-frame is of 32 bits long and consists of various fields as defined in [Table 2-1](#).

Table 2-1 AVSBus Frame Fields

Field Name	Size (Bits)	Description
StartCode	2	Displays the start code sent by the controller indicating the beginning of the new controller sub-frame. This value of this field is 2'b01.
Cmd	2	Determines the type of operation that the controller requires. It can take following values: ■ 2'b11: Read data. ■ 2'b10: Reserved. ■ 2'b01: Write and hold. Writes the data and holds, but does not commit (leave pending). ■ 2'b00: Write and commit. Writes the data and commits all pending writes.
CmdGroup	1	Differentiates between the following two groups of data types: ■ 1'b0: AVSBus data types. ■ 1'b1: Manufacturer-specific data types.
CmdData Type	4	Indicates the type of data to which the command is applicable, that is, the type of data being sent or requested. Following are the data types for command group of AVSBus: ■ 4'b0000: Target rail voltage. ■ 4'b0001: Target rail Vout transition rate. ■ 4'b0010: Rail current (read only). ■ 4'b0011: Rail temperature (read only). ■ 4'b0100: Reset rail voltage to default value (write only). ■ 4'b0101: Rail power mode. ■ 4'b0110 to 4'b1101b: Reserved command data types. ■ 4'b1110: AVSBus Status. ■ 4'b1111: AVSBus Version. For command group of manufacturer-specific data types, you must define the data types.

Table 2-1 AVSBus Frame Fields (continued)

Field Name	Size (Bits)	Description
Select	4	Differentiates between instances of a command data type on a device. ■ For AVSBus data types, this field is a rail selector. ■ For manufacturer-specific data type, this field is specified by the device manufacturer.
CmdData	16	Indicates the data being transferred from controller to target.
CRC	3	Indicates the Cyclic Redundancy Check (CRC) from the controller, which is used for error detection during the transmission of a controller sub-frame.
Reserved	N	Displays a number of unused bits reserved for future use. Reserved bits must be sent as 1s.
TargetAck	2	Indicates the response from a target.
StatusResponse	5	Indicates the status of the transaction from the target.

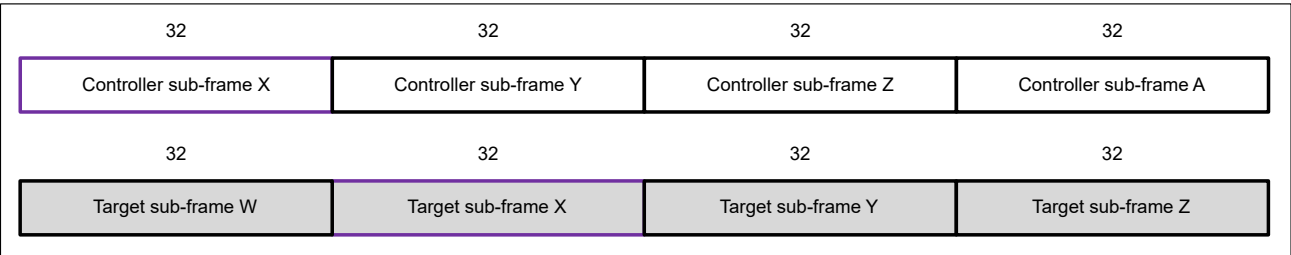
DWC_avsbus transmits the Most Significant Bit (MSB) the first and Least Significant Bit (LSB) the last. The receiver data byte order is also the same – MSB the first and LSB the last. DWC_avsbus controller transmitter launches the data (through the `avsc_cdata` output signal) on the rising edge of the AVSBus clock – `avsc_clk`. DWC_avsbus controller receiver captures the data `avsc_tdata` on the falling edge of the AVSBus clock – `avsc_clk`.

2.3 Frame Alignment

DWC_avsbus is capable of making the most efficient use of bandwidth. Frames can be initiated in complete isolation from each other, sequentially one after the other, or overlapping.

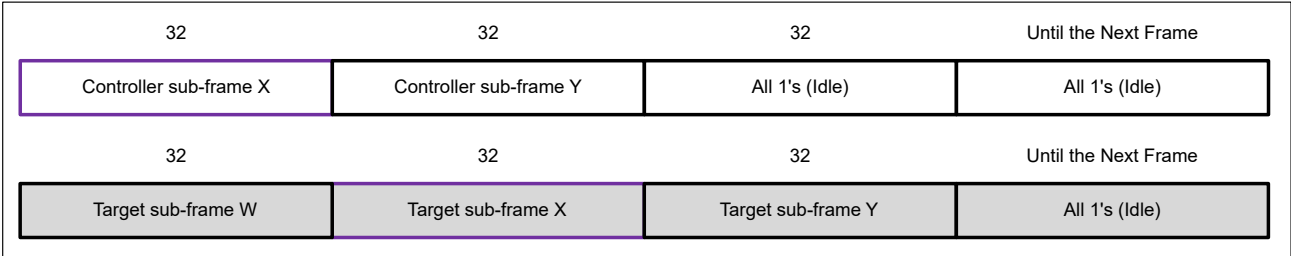
For the most efficient use of bandwidth, DWC_avsbus supports back-to-back frame transfers utilizing the bandwidth. To achieve maximum throughput the transmit FIFO must have multiple entries to be sent over as controller sub-frames. The controller keeps sending the controller sub-frames as long as transmit FIFO has valid command entries to be sent. Parallely, the controller keeps receiving the target sub-frames, thereby making a pipelined mode of operation for maximum throughput. [Figure 2-7](#) represents the frame alignment for this mode of operation. In this mode of operation, the controller is capable of receiving status response frames.

Figure 2-7 Pipeline Transfer Mode for Maximum Throughput



The overlapping of the frame can continue as long as there are more frames for the controller to initiate in the transmit FIFO. When the controller has nothing to send, there is no overlapping frames. Instead, the controller data signal `avsc_cdata` is returned to its default state (logic 1) until the end of the target sub-frame. Then the controller goes idle, and both `avsc_cdata` and `avsc_tdata` signals stay at default value until the start of a new sequence. The end of a sequence happens as shown in [Figure 2-8](#).

Figure 2-8 Last Frame in a Sequence



2.4 Supported Command Data Types

DWC_avsbus supports all commands defined in the AVSBus specification. The command data type is defined in the controller sub-frame as a 4-bit field. DWC_avsbus supports both AVSBus-specific command data types as well as manufacturer-specific data types as defined by the command group field. When the transmit FIFO is filled with controller sub-frames, you must write the 4-bit command data type with either the AVSBus supported command data types or manufacturer-specific command data types. These 4-bit AVSBus-specific command data types are described in [Table 2-2](#). The manufacturer-specific data types and corresponding other field (Select) of the controller sub-frame must be defined by the user.

Table 2-2 Command Data Types

Command Data Type Value	Description
Command Group (CMD_GRP) = 1'b0 (AVSBus-specific Data Types)	
4'b0000	Voltage Read/Write
4'b0001	Vout Transition Rate Read/Write
4'b0010	Current Read
4'b0011	Temperature Read
4'b0100	Voltage Reset
4'b0101	Power Mode Read/Write
4'b0110	Reserved for future use
4'b0111	Reserved for future use
4'b1000	Reserved for future use
4'b1001	Reserved for future use
4'b1010	Reserved for future use
4'b1011	Reserved for future use
4'b1100	Reserved for future use
4'b1101	Reserved for future use
4'b1110	AVSBus Status Read/Write
4'b1111	AVSBus Version Read
Command Group (CMD_GRP) = 1'b1 (Manufacturer-Specific Data Types)	
4'b0000 – 4'b1111	Manufacturer-Specific Read/Write

2.5 AVSBus Clock Generation

DWC_avsbus supports uninterrupted AVSBus clock (avsc_clk) generation. The DWC_avsbus generates the AVSBus clock from the reference clock input – DWC_avsbus core clock (avsc_core_clk). The frequency of the AVSBus clock avsc_clk is derived from the following equation:

$$\text{avsc_clk frequency} = \text{avsc_core_clk frequency} / \text{AVSC_CLK_DIV_REG.CORE_CLK_DIV}$$

The DWC_avsbus suspends the clock whenever there is no transfer required to be sent on the bus, that is, there are no valid commands in the transmit FIFO – it does so when the AVSC_CTL1_REG.IDLE_ST_CLK_SUSP_EN register field is set to 1. The DWC_avsbus Controller keeps the suspended or inactive clock to logic 0 state to ensure that whenever it restarts the clock, the first detectable edge is the rising edge. DWC_avsbus sets the AVSC_STATUS_REG.CLK_SUSPENDED register field indicating that the clock is suspended.

**Note**

DWC_avsbus does not generate AVSBus clock (avsc_clk) when it is disabled (AVSC_EN_REG.ENABLE = 0).

2.5.1 Registers Related to AVSBus Clock Generation Feature

The following register is related to the AVSBus Clock Generation Feature.

- AVSC_CLK_DIV_REG

For more information about this register, see the "Register Description" chapter in the *Reference Manual*.

2.6 Idle Bus Condition

The bus is considered idle in two cases:

- When the clock is suspended between frames.
- When, even in the presence of a clock, no frames are being transmitted.

AVSC_CTL1_REG.IDLE_ST_CLK_SUSP_EN = 1

If DWC_avsbus detects idle state, it suspends the AVSBus clock (avsc_clk output signal) in between the frames and keeps the state of avsc_cdata data signal as high. The controller considers the default state of the avsc_tdata data input signal as logic 1. The controller keeps monitoring the state of avsc_tdata line and alerts the CPU through interrupt avsc_tgt_alert_intr(_n) signal when target changes the avsc_tdata signal state to logic 0. The target must keep avsc_tdata at 0 until the controller initiates a frame.

AVSC_CTL1_REG.IDLE_ST_CLK_SUSP_EN = 0

In this mode, the DWC_avsbus continuously generates the clock. If the controller has not initiated a frame transmission, it keeps the avsc_cdata signal at 1. The controller keeps monitoring the state of avsc_tdata signal and alerts the CPU through the avsc_tgt_alert_intr(_n) interrupt signal when the target changes the avsc_tdata signal to 0. The DWC_avsbus expects the target to keep the signaling state on the avsc_tdata signal to 0 until it initiates a frame.

2.7 Target Acknowledgement

Whenever an AVSBus controller sends a command, the target sends a response, which carries a 2-bit acknowledgement. The 2-bit acknowledgement field indicates the following four states.

- 2'b00: Good CRC, valid data (selector, data type, and data value are good), and resource available. Action taken.
- 2'b01: Good CRC, valid data, but no action is taken due to the unavailability of resource (busy or not allocated to AVSBus).
- 2'b10: Bad CRC. No action taken.
- 2'b11: Good CRC. No action taken. Unable to execute the command due to one of the following conditions:
 - Invalid selector.
 - Invalid data type.
 - Incorrect data.
 - Incorrect action.

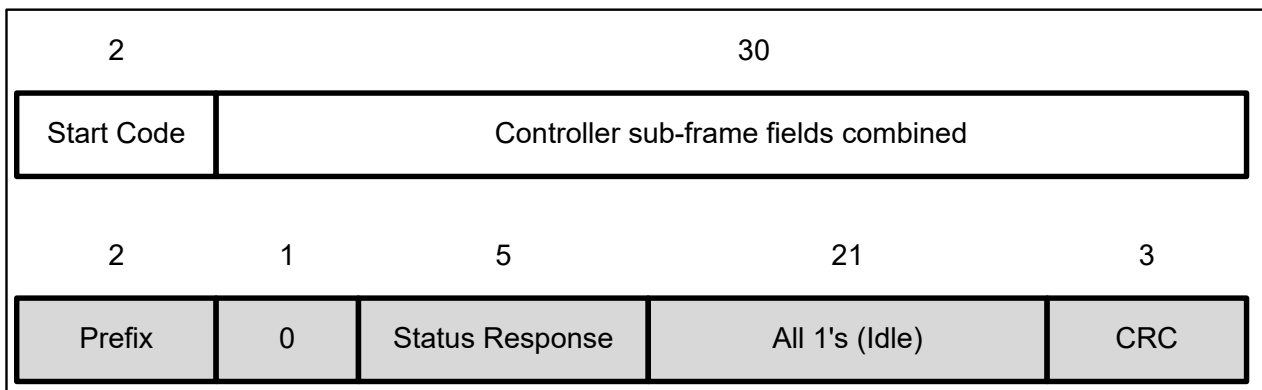
DWC_avsbus controller receives these acknowledgement bits along with the target sub-frame and puts it into the receive FIFO. While CPU reads the receive FIFO content, it should decode the acknowledgement bits and take necessary action on command it sent.

2.8 Status Response Frame

After the AVSBus interface has been idle, if there are more frames to be sent, AVSBus controller issues the first controller sub-frame with the start code. While the first controller sub-frame is being sent by the controller, the target must keep the `avsc_tdata` signal high while it receives the first controller sub-frame in a sequence. However, to take advantage of this bandwidth the AVSBus specification defines a special type of target sub-frame called as Status Response Frame. The purpose of the Status response frame is to bring valuable and timely information to the controller.

The Status Response Frame is structured as shown in [Figure 2-9](#) in the context of a generic Controller sub-frame. The zero at the 3rd bit of the frame is followed by the five bits that make up status response which in turn are followed by all 1's padding to preserve the frame size, and finally a standard 3-bit CRC.

Figure 2-9 Status Response Frame



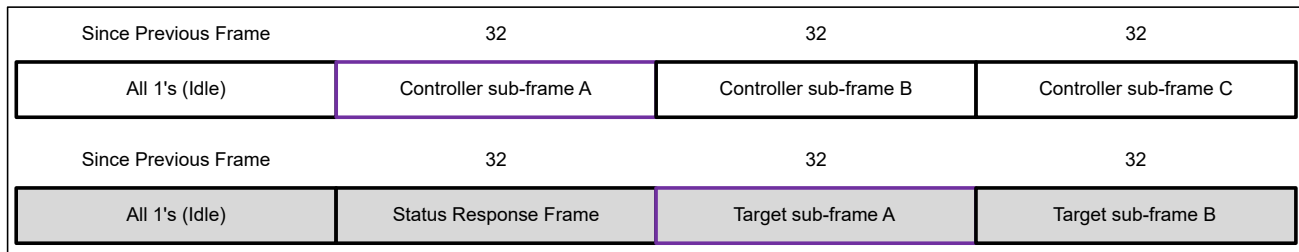
The Prefix is a 2-bit placeholder at the beginning of the frame. Only two values are valid for this prefix.

- If the target signals an interrupt by lowering `avsc_tdata` prior to the appearance of start code, it would naturally be expected to be 2'b00, providing continuity.
- If the target did not signal an interrupt, `avsc_tdata` is at the default high state when start code appears, and it would retain that value, that is, 2'b11.

The `DWC_avsbus` monitors the `avsc_tdata` set low during 3rd bit of the first byte of a Status Response frame. As long as the signal is low during that bit, it accepts Status Response as valid. The format of the Status Response Frame shown in [Figure 2-9](#) is directly compatible with the reply of a target to a command from the controller, with two differences:

- Target replaces the prefix with target acknowledgement when the target is responding to a controller command.
- The all 1's idle field corresponds to the rest of the target's response, which is different for a read frame than for a write.

Incorporating the concept of the Status Response Frame, the first frame in a sequence of overlapping frames would appear as shown in [Figure 2-10](#).

Figure 2-10 First Frame in a Sequence

DWC_avsbus asserts the interrupt `avsc_sts_resp_frame_det_intr(_n)` when Status Response frame is received.

DWC_avsbus has a provision to indicate whether the current frame is a Status Response frame or a target sub-frame. When the `AVSC_CTL1_REG.FRAME_RSVD_BIT_UPDATE_EN` register field is set to 1, DWC_avsbus checks whether the received frame is a Status Response frame or target sub-frame. If it is a Status Response frame, DWC_avsbus sets 3rd bit of last byte (Reserved bit) to 0, so that a Status Response frame can be differentiated from a Target sub-frame. All other bits are unmodified and the updated Status Response frame is pushed into the Receive FIFO.

**Note**

After `avsc_cdata` is idle, DWC_avsbus initiates a new AVSBus transfer if there are at least two spaces available in the RX FIFO. Otherwise, it waits for the space to be available before initiating a new frame.

2.8.1 Interrupt Related to Status Response Frame

The following interrupt indicates status response frame detection.

- `avsc_sts_resp_frame_det_intr(_n)`

For more information about this interrupt, see the "Signals Description" chapter in the *Reference Manual*.

2.9 Target Resynchronization

2.9.1 Overview of Target Resynchronization Feature

An AVSBus target must implement the resynchronization mechanism to recover in case of noise in the line or some other artifact has caused it to reach an incorrect state. Receiving 34 clock pulses while `avsc_cdata` signal is held high causes a target to resynchronize its communication interface, and wait for the next Start Code.

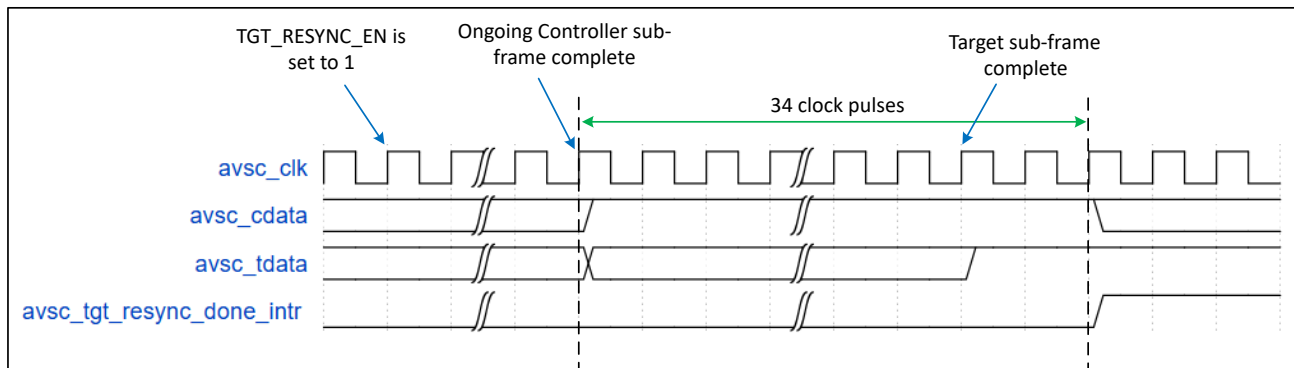
For the resynchronization mechanism to work correctly, the AVSBus target must keep counting ones as long as it is receiving clock pulses, and reset the count whenever it receives a zero.

2.9.2 Description of Target Resynchronization Feature

DWC_avsbus supports Target resynchronization mechanism. Target resynchronization mechanism is software controllable through the `AVSC_CTL2_REG.TGT_RESYNC_EN` register field. Writing 1 into the `AVSC_CTL2_REG.TGT_RESYNC_EN` register field enables the DWC_avsbus to initiate the target resynchronization, which consists of following steps:

1. DWC_avsbus sends 34 clock pulses keeping the transmit data line `avsc_cdata` at logic 1.
2. DWC_avsbus generates the `avsc_tgt_resync_done_intr(_n)` interrupt when it has completed sending 34 clock pulses and the target resynchronization is complete.

Figure 2-11 Target Resynchronization Sequence



Note

- DWC_avsbus auto clears the `AVSC_CTL2_REG.TGT_RESYNC_EN` register field when target resynchronization sequence is complete.
- The controller does not initiate target resynchronization when it is disabled, that is, `AVSC_EN_REG.ENABLE = 0`.
- If target resynchronization is enabled when a transfer is in progress and there are more sub-frames available in the transmit FIFO, DWC_avsbus Controller waits for completion of its current ongoing sub-frame. After the current controller sub-frame is transmitted, controller initiates the target resynchronization sequence. Thereafter, the controller starts sending remaining sub-frames present in the transmit FIFO.

2.9.3 Signals Related to Target Resynchronization Feature

The following signal is related to the Target Resynchronization Feature.

- `avsc_tgt_resync_done_intr`

For more information about this signal, see the "Signals Description" chapter in the *Reference Manual*.

2.10 Cyclic Redundancy Check (CRC) Verification Feature

2.10.1 Overview of CRC Verification Feature

CRC verification ensures the integrity and reliability of data transmitted over the AVSBus by detecting any errors that might have occurred during transmission.

2.10.2 Description of CRC Verification Feature

DWC_avsbus supports CRC verification of the AVSBus frames. DWC_avsbus internally generates the 3-bit CRC on the first 29 bits of the controller sub-frame, target sub-frame, and Status Response frames and then compares it with the last 3 bits.

The AVSC_CTL1_REG.CRC_CHECK_EN register field enables CRC verification on all the frames to be received and transmitted.

If the AVSC_CTL1_REG.CRC_CHECK_EN register field is set to 0, DWC_avsbus

- Does not support any form of CRC verification.
- Transmits and receives all the frames without CRC check at the hardware level.

If the AVSC_CTL1_REG.CRC_CHECK_EN register field is set to 1, DWC_avsbus

- Supports CRC verification and implements CRC generation and error detection logic in the design.
- Performs CRC verification using a 3-bit CRC scheme.

Uses the following polynomial to calculate the 3-bit CRC:

$$\text{CRC}(x) = x^0 + x^1 + x^3$$

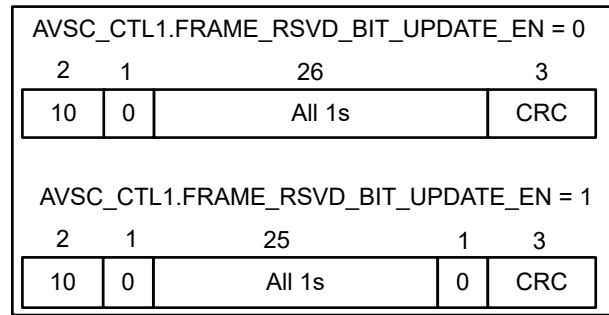
CRC verification of Controller sub-frames

The controller performs the following action when the controller transmit CRC verification fails:

DWC_avsbus pops a command from the transmit FIFO and performs CRC verification before sending it to the target. It initiates a frame if CRC verification has passed on the command popped from the transmit FIFO. In case CRC verification fails, it asserts CRC error interrupt `avsc_ctrlr_sub_crc_err_intr(_n)`.

If the AVSC_CTL1_REG.AVSC_BUF_CRC_FRAME_EN register field is set to 0, DWC_avsbus initiates a frame on the AVSBus interface.

If the register field AVSC_CTL1_REG.AVSC_BUF_CRC_FRAME_EN is set to 1, the controller sub-frame is not sent on the AVSBus interface. For this discarded controller sub-frame, DWC_avsbus controller pushes an entry into its own receive FIFO (on behalf of the target) with the following format.

Figure 2-12 DWC_avsbus CRC Error Target Sub-frame Structure

If the AVSC_CTL1.FRAME_RSVD_BIT_UPDATE_EN register field is set to 0,

- Target acknowledgement is set to “Bad CRC (2'b10)”
- Bit 3 is set to 0
- Remaining bits except CRC field are set to all 1s
- Last 3 bits are set to the CRC value of the updated frame

If the AVSC_CTL1.FRAME_RSVD_BIT_UPDATE_EN register field is set to 1,

- Target acknowledgement is set to “Bad CRC (2'b10)”
- Bit 3 (reserved bit) is set to 0
- 5th bit of last byte (Reserved bit) is set to 0 to indicate that the sub-frame is sent by DWC_avsbus (and not from target)
- Remaining bits except CRC field are set to all 1s
- Last 3 bits are set to the CRC value of the updated frame

Further, DWC_avsbus controller pops a new command from the transmit FIFO and processes it to send as a controller sub-frame.

CRC verification of Target sub-frame/Status response frame

DWC_avsbus performs CRC verification on the target sub-frame before pushing it into the receive FIFO. The target sub-frame is pushed into the receive FIFO irrespective of the result of CRC verification. In the event of CRC validation failure, DWC_avsbus generates a CRC error interrupt (avsc_crc_err_intr(_n)).

If the AVSC_CTL1.FRAME_RSVD_BIT_UPDATE_EN register field is set to 1, DWC_avsbus sets 5th bit of last byte (Reserved bit) to 0, to indicate that the target sub-frame has CRC error. If the AVSC_CTL1.FRAME_RSVD_BIT_UPDATE_EN register field is set to 0, the frame remains unmodified.

DWC_avsbus performs the same steps on the Status Response Frame as well. For more information on Status Response frame, refer section “[Status Response Frame](#)” on page 32.

2.10.3 Pause on CRC Error

DWC_avsbus implements the functionality to stop initiating new frames on the AVSBus interface in the event of CRC verification failure of a controller sub-frame. This helps the software to terminate further frames from getting initiated on the AVSBus interface when a CRC error occurs. This functionality can be enabled through the AVSC_CTL1_REG.PAUSE_ON_CRC_ERR register field.

When the `AVSC_CTL1_REG.PAUSE_ON_CRC_ERR` register field is set to 1, `DWC_avsbus` asserts the `avsc_ctrlr_sub_crc_err_intr(_n)` interrupt if CRC verification fails for the controller sub-frame. It does not issue further transfers on the AVSBus interface until the software services the `avsc_ctrlr_sub_crc_err_intr(_n)` interrupt.

Software must select one of the following ways to deal with the CRC error on the controller sub-frame.

- **Continue with CRC error:** To ignore the CRC error and continue sending the frame with CRC error on the serial interface, software must clear the interrupt `avsc_ctrlr_sub_crc_err_intr(_n)` and write 2'b01 to the `AVSC_CTL2_REG.RESUME_XFER` register field.
- **Discard CRC error frame:** To discard the frame which has the CRC error and continue with subsequent entries of transmit FIFO, software must clear the interrupt `avsc_ctrlr_sub_crc_err_intr(_n)` and write 2'b10 to the `AVSC_CTL2_REG.RESUME_XFER` register field. `DWC_avsbus` discards the current frame which is popped from the transmit FIFO and continues operation on the subsequent entries of the transmit FIFO if it is not empty.
- **Flush transmit FIFO:** To flush the contents of the transmit FIFO, software must clear the interrupt `avsc_ctrlr_sub_crc_err_intr(_n)` and write 1 to the `AVSC_FIFO_FLUSH_REG.TX_FIFO_FLUSH` register field. When this action is performed, `DWC_avsbus` discards the frame with the CRC error and flushes the transmit FIFO.

`DWC_avsbus` does not proceed with further operation until new frames are written to the transmit FIFO.

When the `AVSC_CTL1_REG.PAUSE_ON_CRC_ERR` register field is set to 0, `DWC_avsbus` asserts the CRC error interrupt in case of CRC verification failure for controller sub-frame, target sub-frame or status response frames, but it does not wait for the interrupt to be serviced and continues to issue new AVSBus frames irrespective of CRC error interrupt assertion.



Note

- When the `AVSC_CTL1_REG.CRC_CHECK_EN` register field is set to 0, `BUF_CRC_FRAME_EN`, `FRAME_RSVD_BIT_UPDATE_EN`, and `PAUSE_ON_CRC_ERR` fields must be set to 0. If the software attempts to set these fields to 1 when `CRC_CHECK_EN` is set to 0, `DWC_avsbus` sets them to 0.
- When the `AVSC_CTL1_REG.PAUSE_ON_CRC_ERR` register field is set to 1, CRC verification is done for the target sub-frame and Status Response frames and the status is indicated through the `avsc_tgt_sub_crc_err_intr(_n)` and `avsc_sts_resp_crc_err_intr(_n)` interrupts respectively, but `DWC_avsbus` does not pause on receiving the CRC error for these frames.
- When the `AVSC_CTL1_REG.BUF_CRC_FRAME_EN` register field is set to 1, the `AVSC_CTL1_REG.PAUSE_ON_CRC_ERR` register field must be set to 0. If the software attempts to write 1 to both of these fields, `DWC_avsbus` sets them to 1 and 0 respectively.
- When the `AVSC_CTL1_REG.PAUSE_ON_CRC_ERR` register field is set to 1, if `avsc_ctrlr_sub_crc_err_intr(_n)` interrupt is asserted, software must program the `AVSC_CTL2_REG.RESUME_XFER` register field to the valid value (2'b01 or 2'b10) only after the interrupt is cleared. Programming it in other cases results in an unpredictable behavior.

2.10.4 Signals Related to CRC Verification Feature

The following signals are related to the CRC Verification feature.

- `avsc_ctrlr_sub_crc_err_intr(_n)`
- `avsc_tgt_sub_crc_err_intr(_n)`
- `avsc_sts_resp_crc_err_intr(_n)`

For more information about these signals, see the "Signals Description" chapter in the *Reference Manual*.

2.11 Communication from the AVSBus Target to the AVSBus Controller

AVSBus targets are not capable of becoming a controller. Therefore, a target cannot initiate a communication with the controller. AVSBus specification supports a mechanism for target to issue an interrupt to the AVSBus controller so that the target can send timely status information to the controller. To do so, the target simply pulls the `avsc_tdata` to 0 and signals to the AVSBus controller that it needs a frame to be started.

The controller continuously monitors the `avsc_tdata` input signal when the AVSBus is idle. `DWC_avsbus` sends an interrupt `avsc_tgt_alert_intr(_n)` to the CPU whenever it detects the `avsc_tdata` signal is being pulled low while the bus is idle. It is a responsibility of the CPU to program and push commands into the transmit FIFO of the `DWC_avsbus` as soon as possible. Upon detection of a valid command in the transmit FIFO, `DWC_avsbus` initiates the frame.

As described in “[Propagation Delay Management](#)” on page 41, the controller observes the `avsc_tdata` launched by the target a bit late because of the external delays. Due to these delays, it is possible that, though the target issued the target alert when the AVSBus interface is idle, it might be observed by the controller after it initiates a new frame on the AVSBus interface. In this scenario, `DWC_avsbus` asserts the target alert interrupt while generating AVSBus transfers with the entries available in transmit FIFO. It is up to the software to ignore or service this interrupt.



Note

- The input data line from the target `avsc_tdata` must be held low for at least one `avsc_clk` clock cycle for alert interrupt `avsc_tgt_alert_intr(_n)` to be asserted.
- `DWC_avsbus` detects this alert from the target even when it has stalled the clock output – `avsc_clk`. However, `DWC_avsbus` must be enabled to detect the alert from the target.
- If the `avsc_tdata` signal is pulled low by the target when target resynchronization is in progress, `DWC_avsbus` asserts the `avsc_tgt_alert_intr(_n)` interrupt immediately but it waits for the target resynchronization to complete before issuing new controller sub-frame.

2.11.1 Interrupts Related to Communication from the AVSBus Target to the AVSBus Controller

The following interrupt is related to the communication from the AVSBus target to the AVSBus controller.

- `avsc_tgt_alert_intr(_n)`

For more information about this interrupt, see the "Signal Description" chapter in the *Reference Manual*.

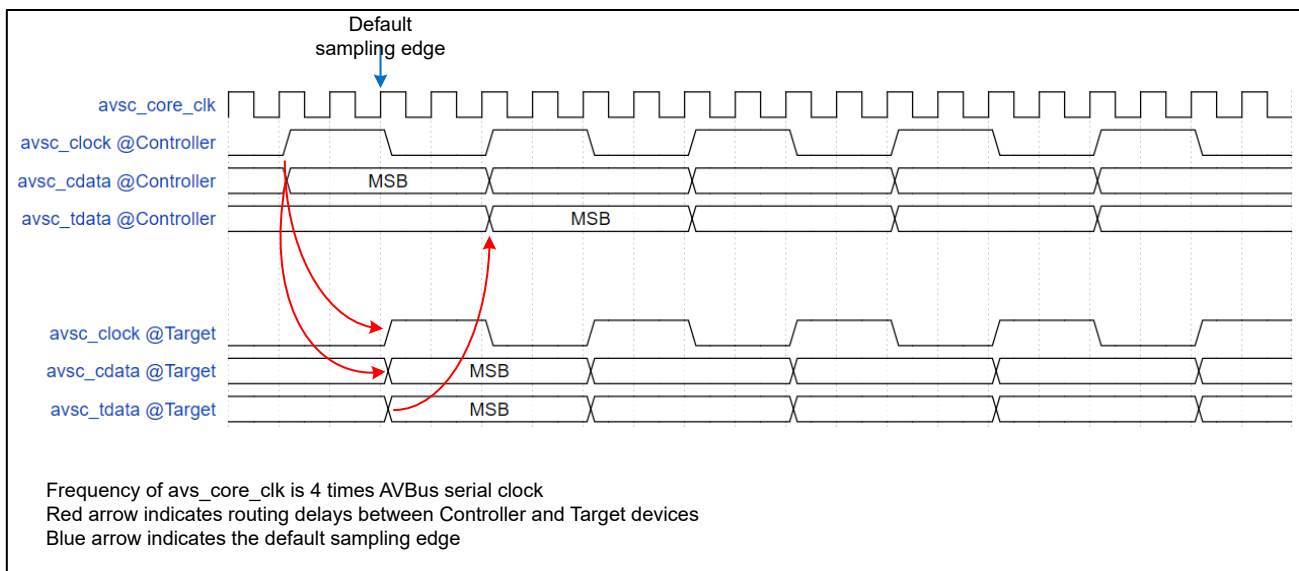
2.12 Propagation Delay Management

In the AVSBus system design, there might be propagation delay of the clock and data from the controller to the target and of the data from the target to the controller due to the following external factors:

- Length of clock and data traces
- Effective load capacitance
- Driver strength

The clock used by the target is a delayed version of the clock in the controller. For this reason, launching data from the target starts later than launching the data from the controller, and capturing by the controller comes earlier. In other words, round trip propagation delays on the AVSBus clock signal from the controller to target and the `avsc_tdata` signal from the target to controller can mean that the timing of the `avsc_tdata` signal – as seen by controller – has moved away from the normal sampling time as shown in [Figure 2-13](#).

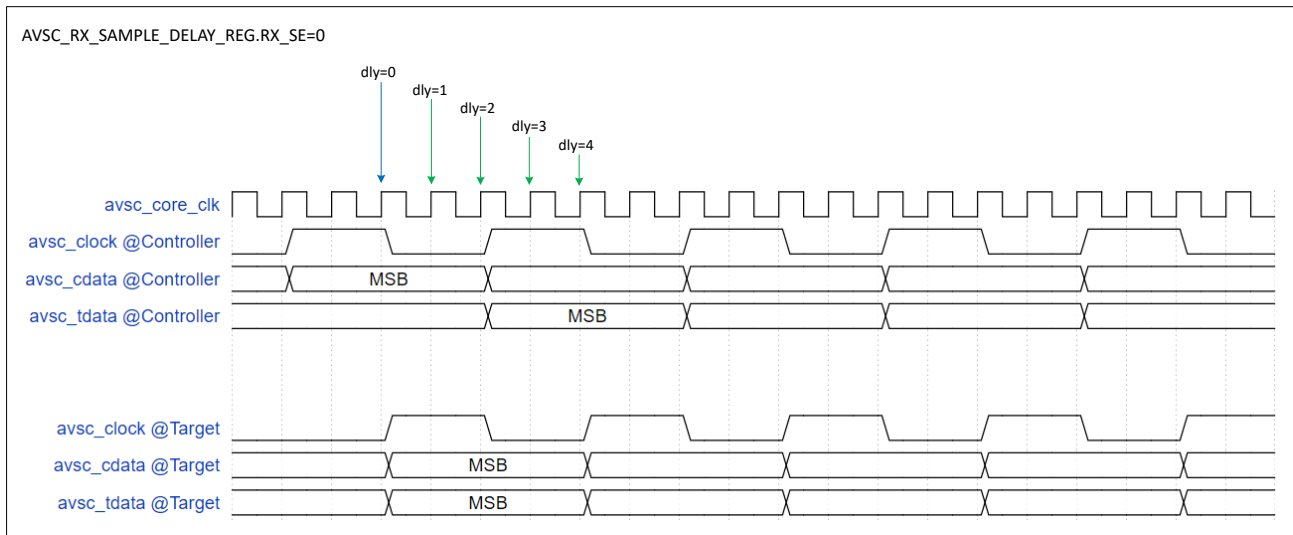
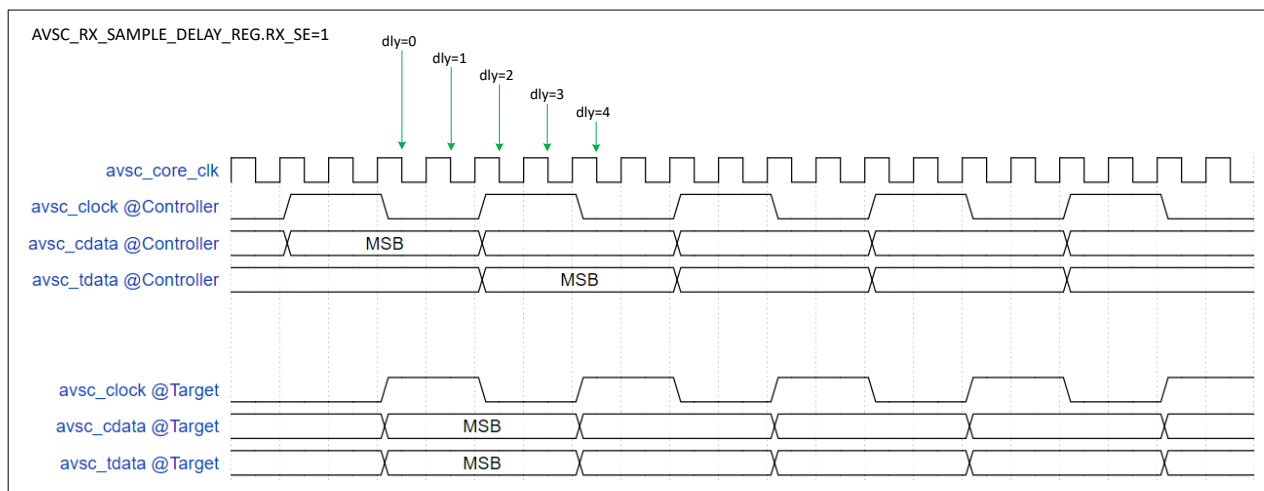
Figure 2-13 Effect of Propagation Delays Between Controller and Target



To compensate for the mentioned propagation delay the `DWC_avsbus` includes additional logic in the design in order to delay the default sampling time of the `avsc_tdata` signal. The `DWC_avsbus` includes a programmable register `AVSC_RX_SAMPLE_DELAY_REG` to dynamically program a delay value in order to move the sampling time of the `avsc_tdata` signal equal to a number of `AVS` core clock (`avsc_core_clk`) cycles from the default sampling edge. The `DWC_avsbus` can be configured to use both positive and negative edges of `avsc_core_clk` to sample the `avsc_tdata` signal by setting the configuration parameter `AVSC_HAS_RX_SAMPLE_DELAY = 2`.

- If the `AVSC_RX_SAMPLE_DELAY_REG.RX_SE` register field is set to 0, then `DWC_avsbus` delays the sampling point by the programmed number of `avsc_core_clk` cycles.
- If the `AVSC_RX_SAMPLE_DELAY_REG.RX_SE` register field is set to 1, then `DWC_avsbus` delays the sampling point by the programmed number of `avsc_core_clk` cycles + $0.5 \times \text{avsc_core_clk period}$, and hence, the sampling is done on the negative edge of the `avsc_core_clk` signal.

[Figure 2-14](#) and [Figure 2-15](#) shows the sampling points delayed with respect to the values programmed in the `AVSC_RX_SAMPLE_DELAY_REG` register.

Figure 2-14 Default and Delayed Sampling Edges on Positive Edge of avsc_core_clk**Figure 2-15 Default and Delayed Sampling Edges on Negative Edge of avsc_core_clk****Note**

- If the `AVSC_RX_SAMPLE_DELAY_REG.RX_SAMPLE_DELAY` register field is programmed to a value greater than the `AVSC_MAX_RX_SAMPLE_DELAY` parameter, `DWC_avsbus` sets this field to `AVSC_MAX_RX_SAMPLE_DELAY`.
- The `AVSC_RX_SAMPLE_DELAY_REG.RX_SE` register field is available only if the configuration parameter `AVSC_HAS_RX_SAMPLE_DELAY` is set to 2; otherwise, `DWC_avsbus` uses the positive edge of `avsc_core_clk` to sample the `avsc_tdata` signal.
- Only the `avsc_tdata` signal sampling logic works on the negative edge of the `avsc_core_clk` signal depending on the programming; the rest of the logic works on the positive edge. `DWC_avsbus` uses `DWC_avsbus_bcm00_ck_mx` module to multiplex between the `avsc_core_clk` and the `~avsc_core_clk`. The multiplexing logic is not glitch-free but it does not create any issue because the logic which uses the output of this multiplexer is not active when there is a possibility of a glitch.

2.12.1 Handling the Propagation Delay

To delay the default sampling time of `avsc_tdata`, select the appropriate option in the **Include RX sample delay logic?** field using the **Top Level Parameters** tab during the **Specify Configuration Activity** in coreConsultant.

- When set to 1, only positive edge of `avsc_core_clk` is used for sampling.
- When set to 2, both positive and negative edges of `avsc_core_clk` are used for sampling.

2.12.2 Registers for Handling Propagation Delay

The following register is related to Propagation Delay handling.

- `AVSC_RX_SAMPLE_DELAY_REG`

For more information about this register, see the "Register Description" chapter in the *Reference Manual*.

2.13 Transmit and Receive FIFO Buffers

DWC_avsbus design adds transmit and receive FIFO buffers to store the controller sub-frames to be sent and the target sub-frames to be received respectively. You can use the configuration parameters `AVSC_TX_FIFO_DEPTH` and `AVSC_RX_FIFO_DEPTH` to configure the depth of these FIFO buffers. You can set the receive FIFO data available and transmit FIFO empty threshold trigger level using the programmable register fields `AVSC_FIFO_TL_REG.RX_FIFO_TL` and `AVSC_FIFO_TL_REG.TX_FIFO_TL` respectively. When this level is reached, DWC_avsbus generates a receive FIFO full interrupt `avsc_rx_fifo_full_intr(_n)`/transmit FIFO empty interrupt `avsc_tx_fifo_empty_intr(_n)`. Actual transmit FIFO empty status is available in `AVSC_STATUS_REG.TFE` register field and actual receive FIFO full status is available in the `AVSC_STATUS_REG.RFF` register field.

Following are the ways to clear these FIFOs:

- Hard reset – through the `presetn` and `avsc_core_rstn` signals.
- Soft reset – by writing 1 to the `AVSC_SOFT_RESET_REG.SOFT_RESET` register field.
- FIFO flush – by writing a 1 to the `AVSC_FIFO_FLUSH_REG.TX_FIFO_FLUSH` and `AVSC_FIFO_FLUSH_REG.RX_FIFO_FLUSH` register fields.
- Disable DWC_avsbus – by writing a 0 to the `AVSC_EN_REG.ENABLE` register field.

Software can access the transmit and receive FIFO buffers through the data port registers. An APB write transfer to `AVSC_DATA_IN_PORT_REG` register accesses the transmit FIFO data and an APB read transfer to `AVSC_DATA_OUT_PORT_REG` register accesses the receive FIFO data.

The transmit FIFO buffer contains the controller sub-frames entries to be transmitted by the DWC_avsbus. Transmit FIFO is updated by an APB write transfer to the data port register `AVSC_DATA_IN_PORT_REG`. [Table 2-3](#) describes the structure of the transmit FIFO entries. The 32-bits APB write data `pdata[31:0]` must have the same structure while writing a controller sub-frame into the transmit FIFO.

The receive FIFO buffer contains the target sub-frames (read or write) or status response frame received by the DWC_avsbus. DWC_avsbus controller pushes these entries into the receive FIFO in the same order that they are received. Target read sub-frame, write sub-frame, and status response frame structures are different. Therefore, when an APB read is performed on the data port register `AVSC_DATA_OUT_PORT_REG`, the 32-bit APB read data `prdata[31:0]` bus value is as described in [Table 2-4](#) for target write sub-frame, read sub-frame, and status response frame respectively.

Table 2-3 Transmit FIFO Buffer Entry Structure

Register Write Data Structure – Bits and Corresponding Fields						
Controller Write Sub-Frame Data Structure						
31:30	29:28	27	26:23	22:19	18:3	2:0
Start Code	Command	Command Group	Command Data Type	Select	Command Data	CRC
Controller Read Sub-Frame Data Structure						
31:30	29:28	27	26:23	22:19	18:3	2:0

Table 2-3 Transmit FIFO Buffer Entry Structure (continued)

Register Write Data Structure – Bits and Corresponding Fields						
Controller Write Sub-Frame Data Structure						
Start Code	Command	Command Group	Command Data Type	Select	Reserved	CRC

Table 2-4 Receive FIFO Buffer Entry Structure

Register Read Data Structure – Bits and Corresponding Fields					
Target Write Sub-Frame Data Structure					
31:29	29	28:24	23:3		2:0
Target ACK	0	Status Response	Reserved		CRC
Target Read Sub-Frame Data Structure					
31:29	29	28:24	23:8	7:3	2:0
Target ACK	0	Status Response	Command Data	Reserved	CRC
Status Response Frame Data Structure					
31:29	29	28:24	23:3		2:0
Prefix	0	Status Response	Reserved		CRC

2.14 Soft Reset Operation

2.14.1 Overview of Soft Reset Operation

DWC_avsbus supports the soft reset feature, which allows the software to reset the controller synchronously without asserting hard reset.

2.14.2 Description of Soft Reset Operation

To soft reset DWC_avsbus, write logic 1 into the AVSC_SOFT_RESET_REG . SOFT_RESET register field.

**Note**

Initiating a soft reset might lead to AVSBus protocol violations on the AVSBus lines because the DWC_avsbus does not make sure that the AVSBus frames initiated or received are completed.

Use soft reset only in the following situations:

- DWC_avsbus stops responding.
- CPU wants to reset DWC_avsbus to recover from the system malfunctions.

CPU writes 1 to the AVSC_SOFT_RESET_REG . SOFT_RESET register field to reset the DWC_avsbus synchronously and the register field is automatically set to 0 after the software reset is complete. When soft reset is initiated, DWC_avsbus performs the following:

- Resets all control registers (except APB).
- Resets all status bits in the interrupt status register.
- Resets all FSM and associated control logic.
- Flushes transmit and receive FIFOs.
- Resets AVSBus interface to its default value.

2.14.3 Registers Related to Soft Reset Operation

The following register is related to the soft reset operation.

- AVSC_SOFT_RESET_REG

For more information about this register, see the "Register Description" chapter in the *Reference Manual*.

2.15 Operation of Interrupt Registers

DWC_avsbus supports interrupt interface and corresponding registers for the CPU attention. It supports two types of interrupt interface based on the value of the configuration parameter `AVSC_INTR_IO_TYPE`.

- `AVSC_INTR_IO_TYPE = 0` (single): In this type, DWC_avsbus supports only a single combined interrupt port `avsc_intr(_n)`.
- `AVSC_INTR_IO_TYPE = 1` (all): In this type, DWC_avsbus supports individual interrupt ports for corresponding events.

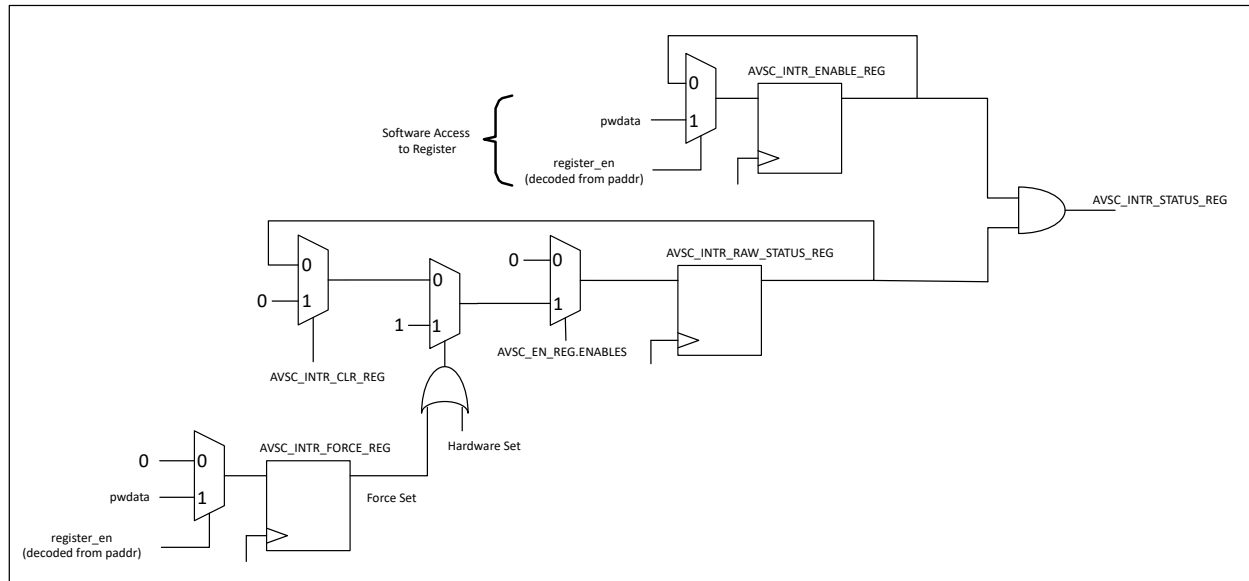
Interrupt polarity is further configurable through the configuration parameter `AVSC_INTR_POL`.

- `AVSC_INTR_POL = 0` (low): All interrupts are active-low signals.
- `AVSC_INTR_POL = 1` (high): All interrupts are active-high signals.

The interrupt status is available in the `AVSC_INTR_STATUS_REG` register. These interrupts can be masked by writing 0 to corresponding fields of the `AVSC_INTR_ENABLE_REG` register. They are cleared by writing into the corresponding fields of the `AVSC_INTR_CLR_REG` register. Unmasked interrupt raw status is available in the `AVSC_INTR_RAW_STATUS_REG` register. The `AVSC_INTR_SOURCE_REG` register contains the source of information for the `avsc_crc_err_intr(_n)` interrupt. All interrupts and register interrupt status bits can be forcefully asserted by writing into the corresponding bits of the `AVSC_INTR_FORCE_REG` register.

Figure 2-16 shows the operation of the interrupt registers where the fields are set by hardware and cleared by software.

Figure 2-16 Interrupt Scheme



2.15.1 Selecting Type and Polarity of Interrupts

To enable the Interrupt Registers, select **All** in the **Select Interrupt Output Port Type** field using the **Top Level Parameters** tab during the **Specify Configuration Activity** in coreConsultant.

To configure the interrupt polarity, select **High** or **Low** in the **Select Polarity of Interrupts?** field using the **Top Level Parameters** tab during the **Specify Configuration Activity** in coreConsultant.

2.15.2 Interrupt Signals

The following signals are related to the Interrupt operation.

- `avsc_intr(_n)`
- `avsc_tx_fifo_empty_intr(_n)`
- `avsc_rx_fifo_full_intr(_n)`
- `avsc_tgt_alert_intr(_n)`
- `avsc_tgt_resync_done_intr(_n)`
- `avsc_ctrlr_sub_crc_err_intr(_n)`
- `avsc_tgt_sub_crc_err_intr(_n)`
- `avsc_sts_resp_crc_err_intr(_n)`
- `avsc_sts_resp_frame_det_intr(_n)`

For more information about these signals, see the "Signals Description" chapter in the *Reference Manual*.

2.15.3 Interrupt Related Registers

The following registers are related to the Interrupt operation.

- `AVSC_INTR_STATUS_REG`
- `AVSC_INTR_ENABLE_REG`
- `AVSC_INTR_CLR_REG`
- `AVSC_INTR_FORCE_REG`
- `AVSC_INTR_RAW_STATUS_REG`

For more information about these registers, see the "Register Description" chapter in the *Reference Manual*.

3

Interfaces

This chapter contains the following sections:

- [“APB Completer Interface”](#) on page 50
- [“DMA Handshaking Interface”](#) on page 54
- [“External Memory Interface”](#) on page 63
- [“Debug Interface”](#) on page 65

3.1 APB Completer Interface

The APB Completer Interface implements the logic to access the internal registers of DWC_avsbus by a host processor (for example a CPU). The APB Completer Interface supports APB4 interface with data bus width of 32-bits. These modules support only the little-endian scheme for data transfer.

The APB Completer Interface can be configured to operate on a different clock from the DWC_avsbus core clock (`avsc_core_clk`), which can be configured using the `AVSC_CLK_MODE` parameter in `coreConsultant`. If `AVSC_CLK_MODE` parameter is set to 1 (Asynchronous), the APB Completer Interface module operates on `pclk`, which can be connected to different clock than `avsc_core_clk`. The DWC_avsbus design takes care of the clock domain crossing.

The APB Completer Interface can be used to perform the read or write operations to the following Register Spaces:

- DWC_avsbus Operational Register Block
- DWC_avsbus Core Register Block

Registers in DWC_avsbus are on the `pclk` domain, but status bits reflect actions that occur in the `avsc_core_clk` domain. Therefore, there is a delay when the `pclk` domain register reflects the activity that occurs on the `avsc_core_clk` clock domain side.

Some registers can be written only when the DWC_avsbus is disabled, or programmed by the `AVSC_EN_REG.ENABLE` register field. Software must not disable DWC_avsbus while DWC_avsbus is active. If DWC_avsbus is disabled when it is in the process of transmitting a controller sub-frame, it completes the current frame (both controller and target sub-frames) and delete the contents of transmit and receive FIFOs. Registers that cannot be written to when the DWC_avsbus is enabled are indicated in [Table 3-2](#) on page 52 in APB Response section.

This section further discusses the additional features of the APB completer interface.



Note

If software attempts to disable the controller (by setting the register field `AVSC_EN_REG.ENABLE = 0`) when an AVSBus transfer is going on, the controller completes the ongoing frame (controller and target sub-frame) before getting disabled.

3.1.1 APB Wait Cycle Insertion

DWC_avsbus uses APB interface `pready` signal to insert wait cycles in the transfers. It is used to insert wait cycles when APB Completer Interface is not ready to complete the APB read or write transfers. The `pready` signal is always kept to its default value that is high for all APB accesses except for the `AVSC_DATA_IN_PORT_REG` and `AVSC_DATA_OUT_PORT_REG` register access.

- `AVSC_DATA_IN_PORT_REG` register access: The `pready` signal is pulled low to stall the APB transaction if the TX FIFO is full and it cannot accept any data. The APB transaction completes and the controller asserts the `pready` if the data is read out of the TX FIFO before the register write timeout happens.
- `AVSC_DATA_OUT_PORT_REG` register access: The `pready` signal is pulled low to stall the APB transaction if RX FIFO is empty and it does not have data to return on the APB read data bus. The APB transaction completes and DWC_avsbus asserts the `pready` signal if the data is received in the RX FIFO before the register read timeout happens.

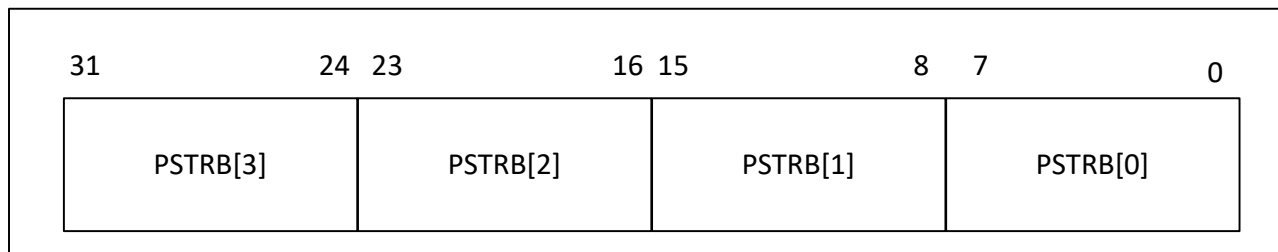
If timeout happens, then the APB transaction is unsuccessful, that is, the RX FIFO is not read or the TX FIFO is not written. The APB transaction must be re-initiated by software for successful completion.

3.1.2 APB Write Strobe

The software uses the APB write strobe signal `pstrb` to indicate the valid byte is being written in an APB write transfer. For a write transaction to the APB interface, the `pstrb` signal indicates validity of `pdata` bytes. `DWC_avsbus` selectively writes to the bytes of the addressed register whose corresponding bit in the `pstrb` signal is high. Bytes that are strobed low by the corresponding `pstrb` fields are not modified. The incoming strobe bits for a read transaction are always zero as per the APB protocol.

Figure 3-1 shows the byte lane mapping of the `pstrb` signal.

Figure 3-1 APB Write Strobe `pstrb` Byte Lane Mapping



3.1.3 APB Protection

`DWC_avsbus` uses the APB4 Completer interface `pprot` signal to support the protection feature. This protection feature is supported on all programmable registers. The protection level register (`AVSC_PROT_LVL_REG`) defines the APB4 protection level, that is the protected registers are updated only if the `pprot` privilege is more than or equal to the protection privilege programmed in the protection level register (`AVSC_PROT_LVL_REG`). Otherwise, APB error response `pslverr` is asserted and the protected register is not updated.

The protection level register `AVSC_PROT_LVL_REG` has default value equal to the configuration parameter `AVSC_PROT_LVL_DFLT_VALUE`. Further, it can be dynamically updated by software. The APB write to update the `AVSC_PROT_LVL_REG` register must have privileged and secure access level of `pprot` value, that is, `pprot[0]` must be 1 indicating a privileged access, `pprot[1]` must be 0 indicating a secure access.

3.1.4 APB Error Response

`DWC_avsbus` uses the APB `pslverr` signal to support error response feature which provides any subordinate error response from register interface (if required). `DWC_avsbus` generates an error response (asserting `pslverr` signal to high) under the following conditions:

- The `AVSC_PROT_LVL_REG` register is accessed with `pprot` values other than privileged and secure.
- Registers protected through APB protection feature (`pprot`) are accessed without appropriate authorization levels. Table 3-1 indicates scenario under which APB subordinate error `pslverr` signal is asserted when register accesses do not meet appropriate authorization levels.

Table 3-1 PPROT Level, Protection Level Programmed in AVSC_PROT_LVL_REG, and APB Error Response

PPROT		AVSC_PROT_LVL_REG		PSLVERR
[1]	[0]	[1]	[0]	
X	0	X	1	HIGH
1	X	0	X	HIGH

- The DWC_avsbus stalls the APB transaction by pulling pready low, as the Receiver FIFO is empty, or transmitter FIFO is full. To avoid locking of the bus for large number of clock cycle, DWC_avsbus provides a timeout option through the AVSC_REG_TIMEOUT_REG register. The timeout is triggered under following conditions:

- Receive FIFO remains empty
- Transmit FIFO remains full

If the duration is equal to the timeout period that is AVSC_REG_TIMEOUT_REG, then APB interface asserts the `pslverr` signal to indicate the register read/write timeout.

- Write to a read-only register.
- Read to a write-only register.
- APB access to an invalid address location.
- Write access to AVSC_DATA_IN_PORT_REG in following cases:
 - With `pstrb` set to values other than 4'b1111.
 - When DWC_avsbus is disabled.
 - When TX FIFO flush (AVSC_FIFO_FLUSH_REG.TX_FIFO_FLUSH) is set to 1.
- Write access to registers when DWC_avsbus is enabled. [Table 3-2](#) lists registers which must not be written when DWC_avsbus is enabled (AVSC_EN_REG.ENABLE = 1). Otherwise, DWC_avsbus triggers an error response. The Controller must be disabled (AVSC_EN_REG.ENABLE = 0) prior to any changes in these registers.

Table 3-2 List of Registers to be Written only when DWC_avsbus is Disabled

Register	Name
AVSC_CTL1_REG	DWC_avsbus Control Register 1.
AVSC_CLK_DIV_REG	DWC_avsbus Core Clock Divisor Register.
AVSC_RX_SAMPLE_DELAY_REG	DWC_avsbus Receiver Sample Delay Register.
AVSC_FIFO_TL_REG	DWC_avsbus FIFO Threshold Level Register.
AVSC_REG_TIMEOUT_REG	DWC_avsbus Register Timeout Value Register.
AVSC_PROT_LVL_REG	DWC_avsbus APB Protection Level Register.

Table 3-2 List of Registers to be Written only when DWC_avsbus is Disabled (continued)

Register	Name
AVSC_DMA_CTL_REG	DWC_avsbus DMA Control Register.
AVSC_DMA_TDL_REG	DWC_avsbus DMA Transmit Data Level Register.
AVSC_DMA_RDL_REG	DWC_avsbus DMA Receive Data Level Register.

3.1.5 Signals Related to APB Completer Interface

The following signals are related to the APB Completer Interface.

- pready
- pwrdata
- pstrb
- pprot
- pslverr
- pclk
- psel
- penable
- presetn
- prdata

For more information about these signals, see the "Signals Description" chapter in the *Reference Manual*.

3.2 DMA Handshaking Interface

The DWC_avsbus has an optional built-in DMA capability that can be selected at the time of configuration; it has a handshaking interface to a DMA controller to request and control transfers. The APB bus is used to perform the data transfer to or from the DMA controller. While the DWC_avsbus DMA operation is designed in a generic way to fit in any DMA controller as easily as possible, it is designed to work seamlessly, and best used, with the DesignWare DMA Controller, the DW_axi_dmac and DW_ahb_dmac. The settings of the DW_axi_dmac that are relevant to the operation of the DWC_avsbus are described here, mainly fields in the DW_axi_dmac channel control register, CHx_CTL, where x is the channel number. Even though following section describes the integration DWC_avsbus with DW_axi_dmac it can be used with the DW_ahb_dmac as well.



Note

When the DWC_avsbus interfaces with the DW_axi_dmac, the DW_axi_dmac is always a flow controller; that is, it controls the block size. This must be programmed by software in the DW_axi_dmac. The DW_axi_dmac always transfers data using DMA burst transactions, if possible, for efficiency. For more information, see the *DesignWare DW_axi_dmac Databook*. Other DMA controllers act in a similar manner.

The relevant DMA settings are discussed in the following sections.



Note

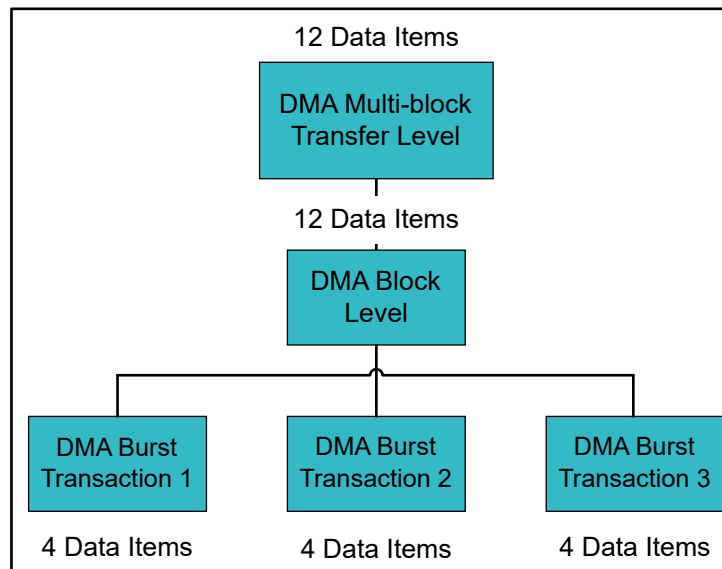
The DMA output dma_finish is a status signal to indicate that the DMA block transfer is complete. DWC_avsbus does not use this status signal, and therefore does not appear in the I/O port list.

3.2.1 Overview of Operation

As a block flow control device, the DMA Controller is programmed by the processor with the number of data items (block size) that are to be transmitted or received by DWC_avsbus; this is programmed into the CHx_BLOCK_TS.BLOCK_TS register field of the DW_axi_dmac.

The block is broken into a number of transactions, each initiated by a request from the DWC_avsbus. The DMA Controller must also be programmed with the number of data items (in this case, DWC_avsbus FIFO entries) to be transferred for each DMA request. This is also known as the burst transaction length and is programmed into the CHx_CTL.SRC_MSIZE/DST_MSIZE register fields of the DW_axi_dmac for source and destination, respectively.

Figure 3-2 shows a single block transfer, where the block size programmed into the DMA Controller is 12 and the burst transaction length is set to 4. In this case, the block size is a multiple of the burst transaction length. Therefore, the DMA block transfer consists of a series of burst transactions. If the DWC_avsbus makes a transmit request to this channel, four data items are written to the DWC_avsbus TX FIFO. Similarly, if the DWC_avsbus makes a receive request to this channel, four data items are read from the DWC_avsbus RX FIFO. Three separate requests must be made to this DMA channel before all 12 data items are written or read.

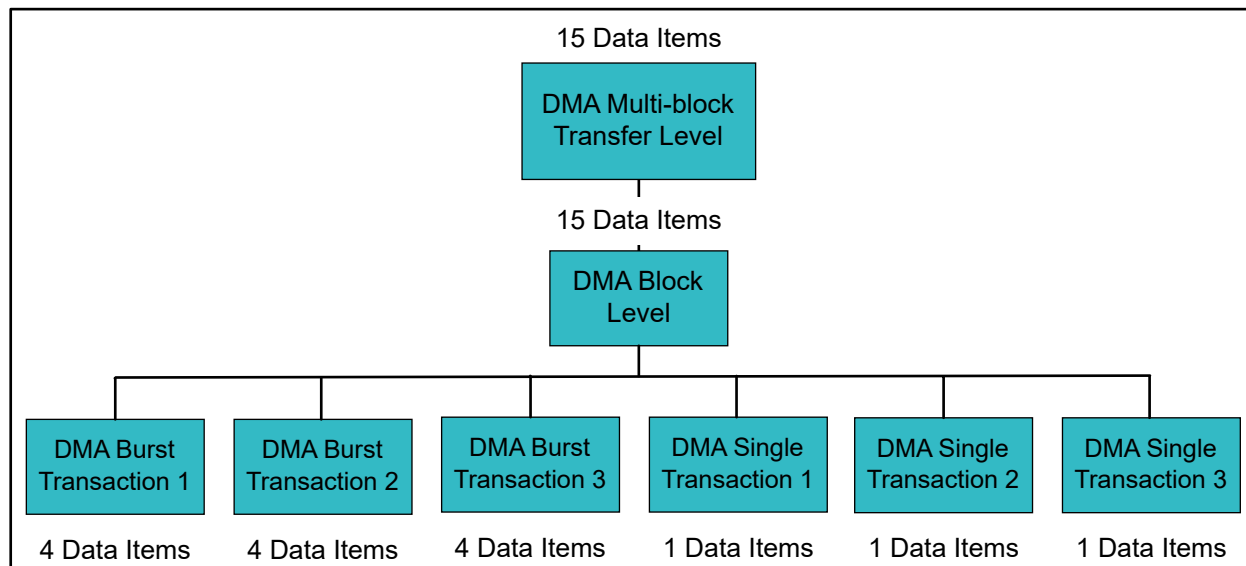
Figure 3-2 Breakdown of DMA Transfer into Burst Transactions

Block Size: $\text{DMA.CHx_BLOCK_TS.BLOCK_TS} = 12$

Number of data items per source transaction: $\text{DMA.CHx_CTL.SRC_MSIZE} = 4$

AVSBus RX FIFO watermark level: $\text{AVSC.AVSC_DMA_RDL_REG} + 1 = \text{DMA.CHx_CTL.SRC_MSIZE} = 4$

When the block size programmed into the DMA Controller is not a multiple of the burst transaction length, as shown in [Figure 3-3](#), a series of burst transactions followed by single transactions are needed to complete the block transfer.

Figure 3-3 Breakdown of DMA Transfer into Single and Burst Transactions

Block Size: $\text{DMA.CHx_BLOCK_TS.BLOCK_TS} = 15$

Number of data items per source transaction: $\text{DMA.CHx_CTL.DST_MSIZE} = 4$

AVSBus RX FIFO watermark level:

$$AVSC.AVSC_DMA_TDL_REG + 1 = DMA.CHx_CTL.DST_MSIZE = 4$$

3.2.2 Transmit Watermark Level and Transmit FIFO Underflow

During DWC_avsbus serial transfers, transmit FIFO requests are made to the DW_axi_dmac whenever the number of entries in the transmit FIFO is less than or equal to the DMA Transmit Data Level Register AVSC_DMA_TDL_REG value; this is known as the watermark level. The DW_axi_dmac responds by writing a burst of data to the transmit FIFO buffer, of length CHx_CTL.DST_MSIZ.

The data should be fetched from the DMA often enough for the transmit FIFO to perform AVSBus serial transfers continuously (if required); that is, when the FIFO begins to empty (almost) another DMA request should be triggered. Otherwise, the FIFO runs out of data. To prevent this condition, you must set the watermark level correctly.

3.2.3 Choosing the Transmit Watermark Level

Consider the example where the assumption is made:

$$DMA.CHx_CTL.DST_MSIZE = AVSC.AVSC_TX_FIFO_DEPTH - AVSC.AVSC_DMA_TDL_REG$$

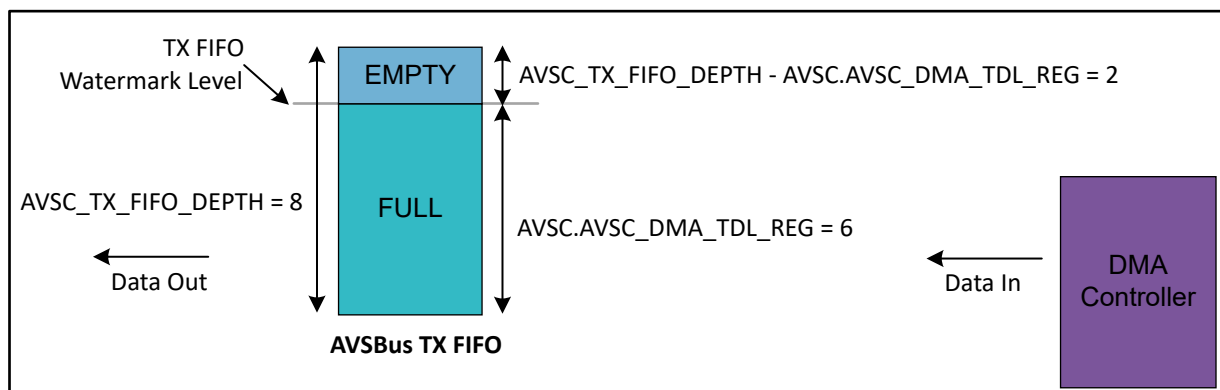
Here the number of data items to be transferred in a DMA burst is equal to the empty space in the Transmit FIFO. Consider two different watermark level settings.

3.2.3.1 Case 1: AVSC_DMA_TDL_REG(TDLR) = 2

- Transmit FIFO watermark level = AVSC.AVSC_DMA_TDL_REG = 2
- DMA.CHx_CTL.DST_MSIZ = AVSC.AVSC_TX_FIFO_DEPTH - AVSC.AVSC_DMA_TDL_REG = 6
- AVSBus transmit FIFO Depth = AVSC.AVSC_TX_FIFO_DEPTH = 8
- DMA.CHx_BLOCK_TS.BLOCK_TS = 30

The above DMA.CHx_CTL.DST_MSIZ programming consideration ensures that DMA never overflows the TX FIFO, as DMA writes the data less than or equal space available in FIFO.

Figure 3-4 TX FIFO - Case 1 Watermark Levels



Therefore, the number of burst transactions needed equals the block size divided by the number of data items per burst:

$$DMA.CHx_BLOCK_TS.BLOCK_TS / DMA.CHx_CTL.DST_MSIZE = 30 / 6 = 5$$

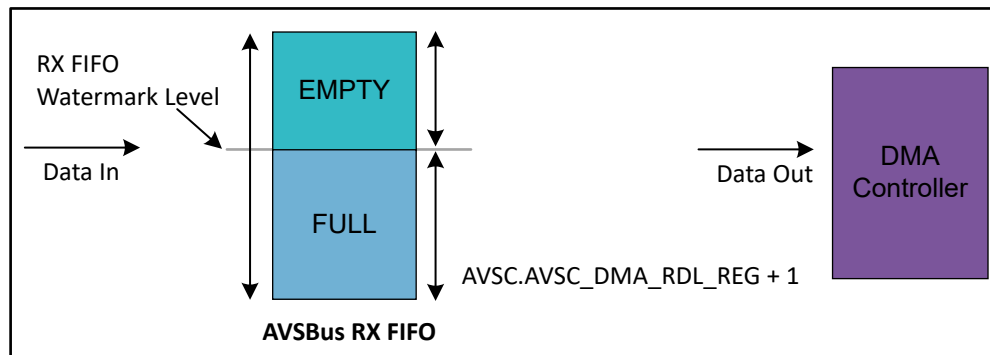
The number of burst transactions in the DMA block transfer is 5. But the watermark level, `AVSC.AVSC_DMA_TDL_REG`, is quite low. Therefore, there is a probability of transmit FIFO becomes empty and AVSBus serial transmission may stop. This occurs because the DMA has not had time to service the DMA request before the transmit FIFO becomes empty. This provides a potentially less amount of AMBA bursts per block and better bus utilization than the case 2.

3.2.3.2 Case 2: IC_DMA_TDLR = 6

- Transmit FIFO watermark level = `AVSC.AVSC_DMA_TDL_REG` = 6
- `DMA.CHx_CTL.DST_MSIZ` = `AVSC.AVSC_TX_FIFO_DEPTH` - `AVSC.AVSC_DMA_TDL_REG` = 2
- AVSBus transmit FIFO Depth = `AVSC.AVSC_TX_FIFO_DEPTH` = 8
- `DMA.CHx_BLOCK_TS.BLOCK_TS` = 30

The above `DMA.CHx_CTL.DST_MSIZ` programming consideration ensures that DMA will never overflow the TX FIFO, as DMA will write the data less than or equal space available in FIFO.

Figure 3-5 TX FIFO - Case 2 Watermark Levels



Number of burst transactions in Block:

$$\text{DMA.CHx_BLOCK_TS.BLOCK_TS} / \text{DMA.CHx_CTL.DST_MSIZ} = 30 / 2 = 15$$

In this block transfer, there are 15 destination burst transactions in a DMA block transfer. But the watermark level, `AVSC.AVSC_DMA_TDL_REG` (TDLR), is high. Therefore, there is a low probability of the transmit FIFO becoming empty, and in turn, AVSBus serial transmission will continue without any obstruction.

Thus, the second case has a lower probability of under supply of data at the expense of more burst transactions per block. This provides a potentially greater amount of AMBA bursts per block and worsen bus utilization than the former case.

Therefore, the goal in choosing a watermark level is to minimize the number of transactions per block, while at the same time keeping the probability of an underflow condition to an acceptable level. In practice, this is a function of the ratio of the rate at which the AVSBus transmits data to the rate at which the DMA can respond to destination burst requests.

For example, promoting the channel to the highest priority channel in the DMA, and promoting the DMA manager interface to the highest priority manager in the AMBA layer, increases the rate at which the DMA controller can respond to burst transaction requests. This in turn allows you to decrease the watermark level, which improves bus utilization without compromising the probability of an underflow occurring.

3.2.4 Selecting DST_MSIZ and Transmit FIFO Overflow

As can be seen from [Figure 3-4](#) on page 56, programming `DMA.CHx_CTL.DST_MSIZ` to a value greater than the watermark level that triggers the DMA request may cause overflow when there is not enough space in the AVSBus transmit FIFO to service the destination burst request. Therefore, the following equation must be adhered to in order to avoid overflow:

$$\text{DMA.CHx_CTL.DST_MSIZ} \leq \text{AVSC.AVSC_TX_FIFO_DEPTH} - \text{AVSC.AVSC_DMA_TDL_REG} \quad (1)$$

In Case 2: `IC_DMA_TDLR = 6`, the amount of space in the transmit FIFO at the time the burst request is made is equal to the destination burst length, `DMA.CHx_CTL.DST_MSIZ`. Thus, the transmit FIFO may be full, but not overflowed, at the completion of the burst transaction.

Therefore, for optimal operation, `DMA.CHx_CTL.DST_MSIZ` should be set at the FIFO level that triggers a transmit DMA request; that is:

$$\text{DMA.CHx_CTL.DST_MSIZ} = \text{AVSC.AVSC_TX_FIFO_DEPTH} - \text{AVSC.AVSC_DMA_TDL_REG} \quad (2)$$

Adhering to equation (2) reduces the number of DMA bursts needed for a block transfer, and this in turn improves AMBA bus utilization.



Note

The transmit FIFO is not full at the end of a DMA burst transfer if the AVSBus controller has successfully transmitted one data item or more on the AVSBus serial transmit line during the transfer.

3.2.5 Receive Watermark Level and Receive FIFO Overflow

During `DWC_avsbus` serial transfers, receive FIFO requests are made to the `DW_axi_dmac` whenever the number of entries in the receive FIFO is at or above the DMA Receive Data Level Register; that is, `AVSC.AVSC_DMA_RDL_REG (RDLR)+1`. This is known as the watermark level. The `DW_axi_dmac` responds by fetching a burst of data from the receive FIFO buffer of length `DMA.CHx_CTL.SRC_MSIZ`.

Data should be fetched by the DMA often enough for the receive FIFO to accept serial transfers continuously; that is, when the FIFO begins to fill, another DMA transfer is requested. Otherwise, the FIFO fills with data (overflow). To prevent this condition, you must correctly set the watermark level.

3.2.6 Choosing the Receive Watermark level

Similar to choosing the transmit watermark level described earlier, the receive watermark level, `AVSC.AVSC_DMA_RDL_REG+1`, should be set to minimize the probability of overflow, as shown in [Figure 3-6](#) on page 59. It is a trade-off between the number of DMA burst transactions required per block versus the probability of an overflow occurring.

3.2.7 Selecting SRC_MSIZ and Receive FIFO Underflow

As shown in [Figure 3-5](#) on page 57, programming a source burst transaction length greater than the watermark level may cause underflow when there is not enough data to service the source burst request. Therefore, equation 3 following must be adhered to avoid underflow.

If the number of data items in the receive FIFO is equal to the source burst length at the time the burst request is made – `DMA.CHx_CTL.SRC_MSIZ` – the receive FIFO may be emptied, but not underflowed, at the completion of the burst transaction. For optimal operation, `DMA.CHx_CTL.SRC_MSIZ` should be set at the watermark level; that is:

$$\text{DMA.CHx_CTL.SRC_MSIZ} = \text{AVSC.AVSC_DMA_RDL_REG} + 1 \quad (3)$$

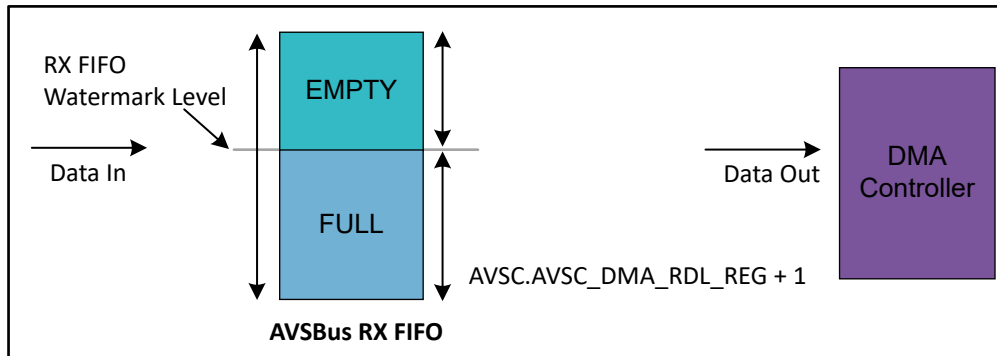
Adhering to equation (3) reduces the number of DMA bursts in a block transfer, which in turn can avoid underflow and improve AMBA bus utilization.



Note

The receive FIFO is not empty at the end of the source burst transaction if the I2C has successfully received one data item or more on the AVSBus serial receive line during the burst.

Figure 3-6 RX FIFO - Watermark Levels



3.2.8 Handshaking Interface Operation

The following sections discuss the handshaking interface.

To enable the DMA Controller interface on the DWC_avsbus, you must write the DMA Control Register (AVSC_DMA_CTL_REG). Writing a 1 into the AVSC_DMA_CTL_REG.TX_DMA_EN register field enables the DWC_avsbus transmit handshaking interface. Writing a 1 into the AVSC_DMA_CTL_REG.RX_DMA_EN register field enables the DWC_avsbus receive handshaking interface.

You can enable the DMA Handshaking Interface using the AVSC_HAS_DMA parameter.

3.2.8.1 dma_tx_req, dma_rx_req

The request signals for source and destination, dma_tx_req and dma_rx_req, are activated when their corresponding FIFOs reach the watermark levels as discussed earlier.

The DW_axi_dmac uses rising-edge transition of the dma_tx_req signal/dma_rx_req to identify a request on the channel. Upon reception of the dma_tx_ack/dma_rx_ack signal from the DW_axi_dmac to indicate the burst transaction is complete, the DWC_avsbus de-asserts the burst request signals, dma_tx_req/dma_rx_req, then dma_tx_ack/dma_rx_ack is de-asserted by the DW_axi_dmac.

When the DWC_avsbus samples that dma_tx_ack/dma_rx_ack is de-asserted, it can re-assert the dma_tx_req/dma_rx_req of the request line if their corresponding FIFOs exceed their watermark levels (back-to-back burst transaction). If this is not the case, the DMA request lines remain de-asserted.

Figure 3-7 shows a timing diagram of a burst transaction where pclk and dmac_core_c clock is asynchronous.

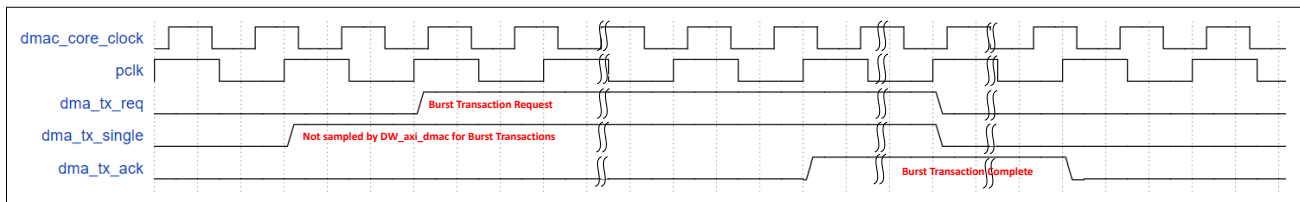
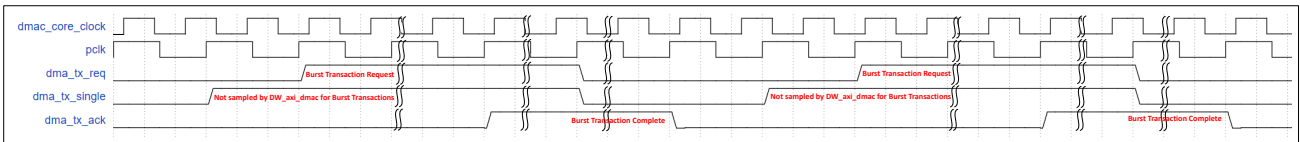
Figure 3-7 Burst Transaction - pclk and dmac_core_clock Asynchronous

Figure 3-8 shows two back-to-back burst transactions where pclk and dmac_core_clock is asynchronous.

Figure 3-8 Back-to-Back Burst Transactions - pclk and dmac_core_clock Asynchronous**Note**

The asynchronous clock domain handling from pclk to dmac_core_clock and vice-versa is handled by DW_axi_dmac. There is no additional consideration required in DWC_avsbus.

The handshaking loop is as follows:

- dma_tx_req/dma_rx_req asserted by DWC_avsbus
- dma_tx_ack/dma_rx_ack asserted by DW_axi_dmac
- dma_tx_req/dma_rx_req de-asserted by DWC_avsbus
- dma_tx_ack/dma_rx_ack de-asserted by DW_axi_dmac

dma_tx_req/dma_rx_req reasserted by DWC_avsbus, if back-to-back transaction is required.

**Note**

The burst transaction request signals, dma_tx_req and dma_rx_req, are generated in the DWC_avsbus pclk and sampled in the DW_axi_dmac by dmac_core_clock. The acknowledge signals, dma_tx_ack and dma_rx_ack, are generated in the DW_axi_dmac of dmac_core_clock and sampled in the DWC_avsbus of pclk. The handshaking mechanism between the DW_axi_dmac and the DWC_avsbus supports quasi-synchronous/asynchronous clocks.

Two things to note here:

1. The burst request lines, dma_tx_req signal/dma_rx_req, once asserted remain asserted until their corresponding dma_tx_ack/dma_rx_ack signal is received even if the respective FIFO's drop below their watermark levels during the burst transaction.
2. The dma_tx_req/dma_rx_req signals are de-asserted when their corresponding dma_tx_ack/dma_rx_ack signals are asserted, even if the respective FIFOs exceed their watermark levels.

3.2.8.2 dma_tx_single, dma_rx_single

The `dma_tx_single` signal is a status signal. It is asserted when there is at least one free entry in the transmit FIFO and cleared when the transmit FIFO is full. The `dma_rx_single` signal is a status signal. It is asserted when there is at least one valid entry in the receive FIFO and cleared when the receive FIFO is empty.

These signals are needed by only the `DW_axi_dmac` for the case where the block size, `CHx_BLOCK_TS.BLOCK_TS`, that is programmed into the `DW_axi_dmac` is not a multiple of the burst transaction length, `CHx_CTL.SRC_MSIZE`, `CHx_CTL.DST_MSIZE`, as shown in [Figure 3-3](#) on page 55. In this case, the DMA single outputs inform the `DW_axi_dmac` that it is still possible to perform single data item transfers, so it can access all data items in the transmit/receive FIFO and complete the DMA block transfer. The DMA single outputs from the `DWC_avsbus` are not sampled by the `DW_axi_dmac` otherwise. This is illustrated in the following example.

Consider first an example where the receive FIFO channel of the `DWC_avsbus` is as follows:

`DMA.CHx_CTL.SRC_MSIZE = AVSC.AVSC_DMA_RDL_REG + 1 = 4`

`DMA.CHx_BLOCK_TS.BLOCK_TS = 12`

For the example in [Figure 3-2](#) on page 55, with the block size set to 12, the `dma_rx_req` signal is asserted when four data items are present in the receive FIFO. The `dma_rx_req` signal is asserted three times during the `DWC_avsbus` serial transfer, ensuring that all 12 data items are read by the `DW_axi_dmac`. All DMA requests read a block of data items and no single DMA transactions are required. This block transfer is made up of three burst transactions.

Now, for the following block transfer:

`DMA.CHx_CTL.SRC_MSIZE = AVSC.AVSC_DMA_RDL_REG + 1 = 4`

`DMA.CHx_BLOCK_TS.BLOCK_TS = 15`

The first 12 data items are transferred as already described using three burst transactions. But when the last three data frames enter the receive FIFO, the `dma_rx_req` signal is not activated because the FIFO level is below the watermark level. The `DW_axi_dmac` samples `dma_rx_single` and completes the DMA block transfer using three single transactions. The block transfer is made up of three burst transactions followed by three single transactions.

[Figure 3-9](#) shows a single transaction. The handshaking loop is as follows:

- `dma_tx_single/dma_rx_single` asserted by `DWC_avsbus`
- `dma_tx_ack/dma_rx_ack` asserted by `DW_axi_dmac`
- `dma_tx_single/dma_rx_single` de-asserted by `DWC_avsbus`
- `dma_tx_ack/dma_rx_ack` de-asserted by `DW_axi_dmac`

Figure 3-9 Single Transaction - `pclk` and `dmac_core_clock` Asynchronous

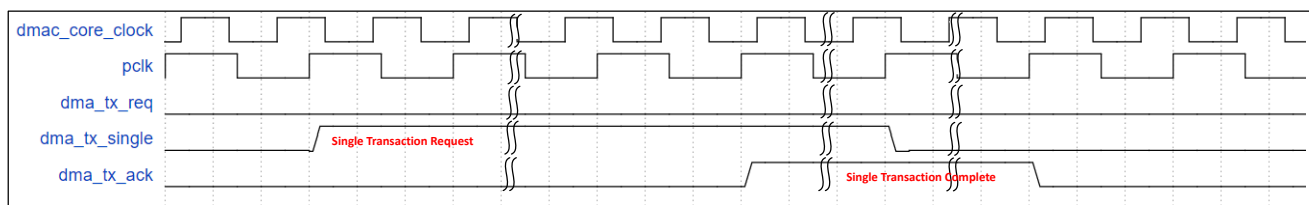
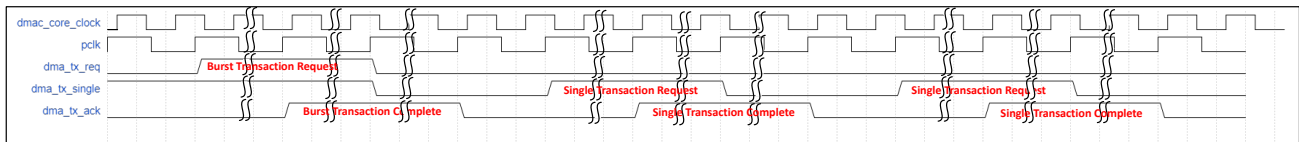


Figure 3-10 shows a burst transaction, followed by three back-to-back single transactions, where pclk and dmac_core_clock is asynchronous.

Figure 3-10 Burst Transaction with 3 Back-to-back Single Transactions - pclk and dmac_core_clock Asynchronous



Note

The single transaction request signals, dma_tx_single and dma_rx_single, are generated in the DWC_avsbus on the pclk and sampled in DW_axi_dmac on dmac_core_clock. The acknowledge signals, dma_tx_ack and dma_rx_ack, are generated in the DW_axi_dmac on the dmac_core_clock edge and sampled in the DWC_avsbus on pclk. The handshaking mechanism between the DW_axi_dmac and the DWC_avsbus supports quasi-synchronous/asynchronous clocks.

3.2.9 Enabling the DMA Handshaking Interface

To enable the DMA handshaking interface, select **Enable** in the **DMA Handshaking Interface** field using the **Top Level Parameters** tab during the **Specify Configuration Activity** in coreConsultant.

3.2.10 Signals Related to DMA Handshaking Interface

The following signals are related to the DMA Handshaking interface.

- dma_tx_ack
- dma_tx_req
- dma_rx_ack
- dma_tx_single
- dma_rx_req
- dma_rx_single

For more information about these signals, see the "Signals Description" chapter in the *Reference Manual*.

3.2.11 Registers Related to DMA Handshaking Interface

The following registers are related to the DMA Handshaking interface.

- AVSC_DMA_CTL_REG
- AVSC_DMA_TDL_REG
- AVSC_DMA_RDL_REG

For more information about these registers, see the "Registers Description" chapter in the *Reference Manual*.

3.3 External Memory Interface

DWC_avsbus transmit and receive FIFO uses flip-flop-based memory. If a flip-flop-based memory is used when the memory dimension is large, it is not area and power efficient. External Memory interface optimizes the FIFO memory with external memories such as SRAM, Register File and so on for area and power optimization.

The QoR of the DWC_avsbus can be improved significantly by implementing FIFO memory with SRAM, or Register File based memories for larger memory dimensions. This feature provides separate external memory interface to connect SRAM or Register File based memories to the transmit and receive FIFOs, when the configuration parameter AVSC_HAS_EXT_MEM is set to 1. External Memory Interface also allows you to connect to the flip-flop-based memory through this interface. This is beneficial when the Channel FIFO memory dimensions are small, and flip-flop-based memories are more QoR efficient than SRAM or Register File based memories.

The read and write operations on the FIFOs can work in parallel. Hence, two-port Register file, SRAM, or flip-flop memory can be connected to the External Memory interface based on the QoR requirement. Generally, Register File or SRAM uses synchronous read timing, and read data is valid in the next cycle after read enable is active. Different FIFO memory types can be used as mentioned earlier. For example, flip-flop-based memory can be used for smaller memory dimensions; and Register File or SRAM can be used for larger memory dimensions. However, no matter which type is adopted, synchronous read timing must be used.

3.3.1 Enabling External Memory

To select transmit and receive FIFO memory as external memory, select **Yes** in the **Include External Memory Interface** Field using the **Top Level Parameters** tab during the **Specify Configuration Activity** in coreConsultant.APB Completer Interface

3.3.2 Signals Related to External Memory Interface

The following signals are related to the External Memory Interface.

- avsc_tx_fifomem_rdata
- avsc_tx_fifomem_wcen
- avsc_tx_fifomem_wen
- avsc_tx_fifomem_waddr
- avsc_tx_fifomem_wdata
- avsc_tx_fifomem_rcen
- avsc_tx_fifomem_ren
- avsc_tx_fifomem_raddr
- avsc_rx_fifomem_wcen
- avsc_rx_fifomem_wen
- avsc_rx_fifomem_waddr
- avsc_rx_fifomem_wdata
- avsc_rx_fifomem_rcen

- `avsc_rx_fifomem_ren`
- `avsc_rx_fifomem_raddr`

For more information about these signals, see the "Signals Description" chapter in the *Reference Manual*.

3.4 Debug Interface

DWC_avsbus supports debug interface to provide in-circuit debugging functionality in the DWC_avsbus of AVSBus subsystem. DWC_avsbus includes the debug signals on its interface when the configuration parameter AVSC_HAS_DEBUG_INTF = 1.

3.4.1 Signals Related to Debug Interface Feature

The following signals are related to the Debug Interface Feature.

- debug_avsc_en
- debug_avsc_busy
- debug_avsc_state
- debug_tx_frame
- debug_rx_frame
- debug_txfifo_level
- debug_rxfifo_level
- debug_avsc_clk_suspended

For more information, refer to the "Signals Description" chapter in the *Reference Manual*.

A

Area and Power

Table A-1 provides information about the synthesis results (power consumption, frequency and area) and DFT coverage of the DWC_avsbus using the industry standard 5nm technology library.

Table A-1 Synthesis Results for AVSbus Controller

Configuration	Operating Frequency	Gate Count	Power Consumption		TetraMax Coverage (%)		SpyGlass StuckAtCov (%)
			Static Power	Dynamic Power	StuckAtTest	Transition	
Prime Profile = AVSBUS	pclk=100 MHz avsc_core_clk=100 MHz	15527	0.0002 mW	0.103 mW	99.96	95.33	99.8